Pontificia Universidad Católica del Perú
Facultad de Ciencias e Ingeniería
Departamento Académico de Ingeniería



1INF02 APRENDIZAJE DE MÁQUINA

Métodos de Ensamble: Bagging, Boosting y Random Forests

Prof. Rodrigo Uribe

Organización de las Clases

(10 min) Preguntas y consultas.

(30 min) Respuesta *Vibe Quest.*

(45 min) Arboles de decisión y descomposición de la varianza y el sesgo.



(10 min) Break

(45 min) Métodos de ensambles: Bagging, Boosting y Random Forests.

(10 min) Ejercicios matemáticos.

(10 min) Preguntas *Vibe Quest.*



Respuestas Vibe Quest

- ¿De qué manera el kernel trick, al mapear implícitamente a espacios de alta dimensión, podría estar enmascarando relaciones genuinas en el espacio original, y cuándo esta "abstracción matemática" se convierte en una distorsión problemática de la realidad subyacente?
- Crea escenarios específicos donde comparemos SVM vs Perceptrón: ¿puedes simular datasets donde el perceptrón sea "suficientemente bueno" versus casos donde la optimización del SVM sea crucial? Exploremos cuándo el costo computacional extra realmente vale la pena.
- Genera ejemplos concretos donde maximizar el margen sea contraproducente: ¿qué pasa con outliers, datos desbalanceados, o distribuciones multimodales? Crear visualizaciones que muestren cuándo el "mejor" separador según SVM no es el más útil en práctica.
- ¿Es el SVM una ruptura fundamental con la estadística tradicional de modelar distribuciones probabilísticas, al reducir toda la incertidumbre a una decisión binaria determinística basada en distancias geométricas? ¿Cuál es el trade-off?

¿Qué modelos han aprendido hasta ahora?

¿Existe un modelo perfecto?

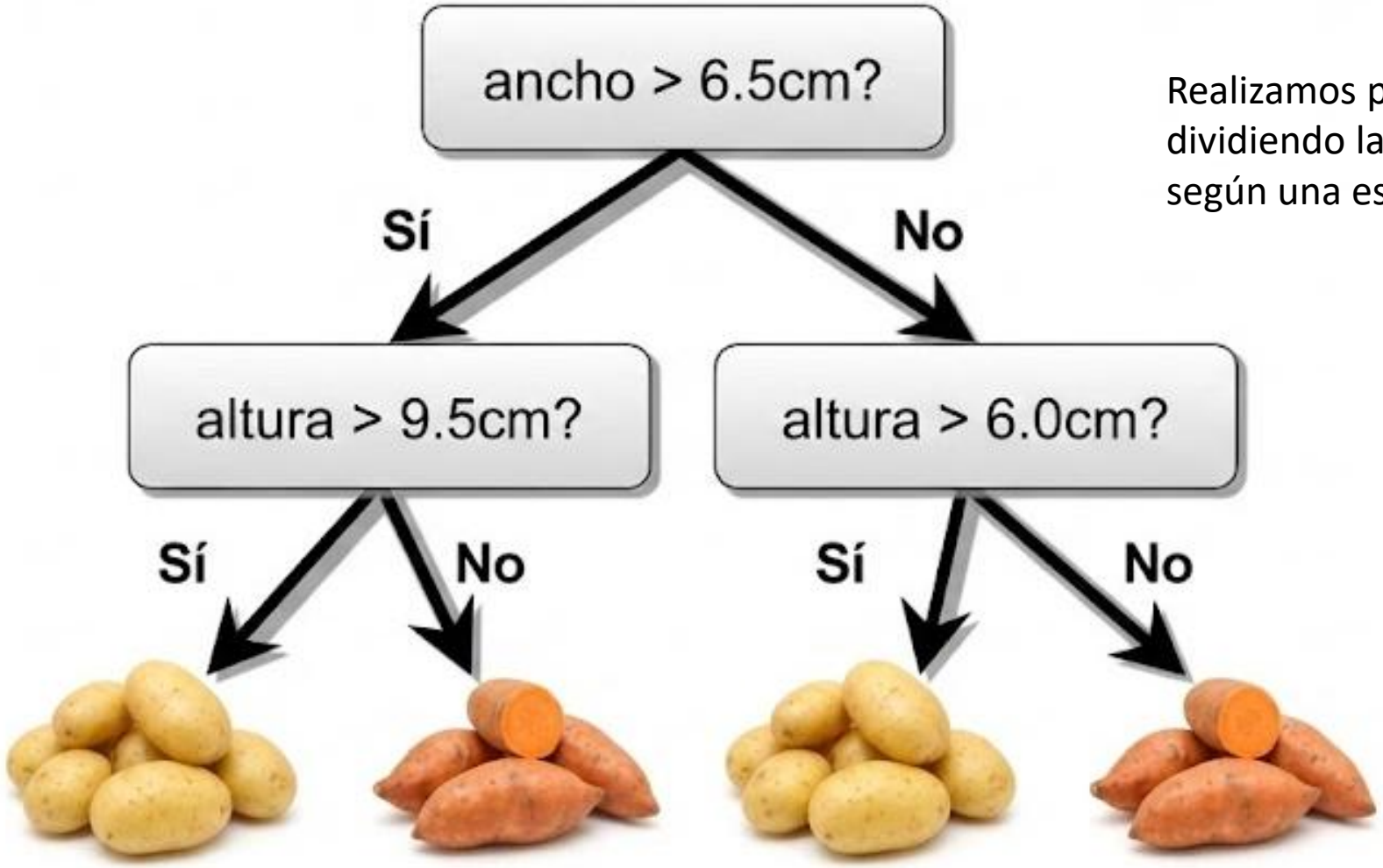
Mezclando papas con camotes



Clasificación Binaria: papas o camotes

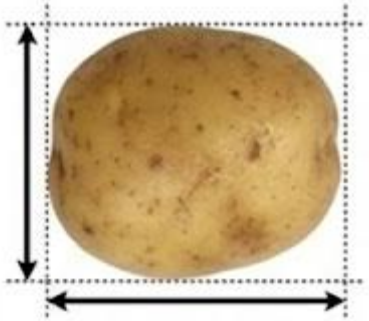
¿Qué características pueden servir?
¿Cómo podemos evitar mezclarlos?

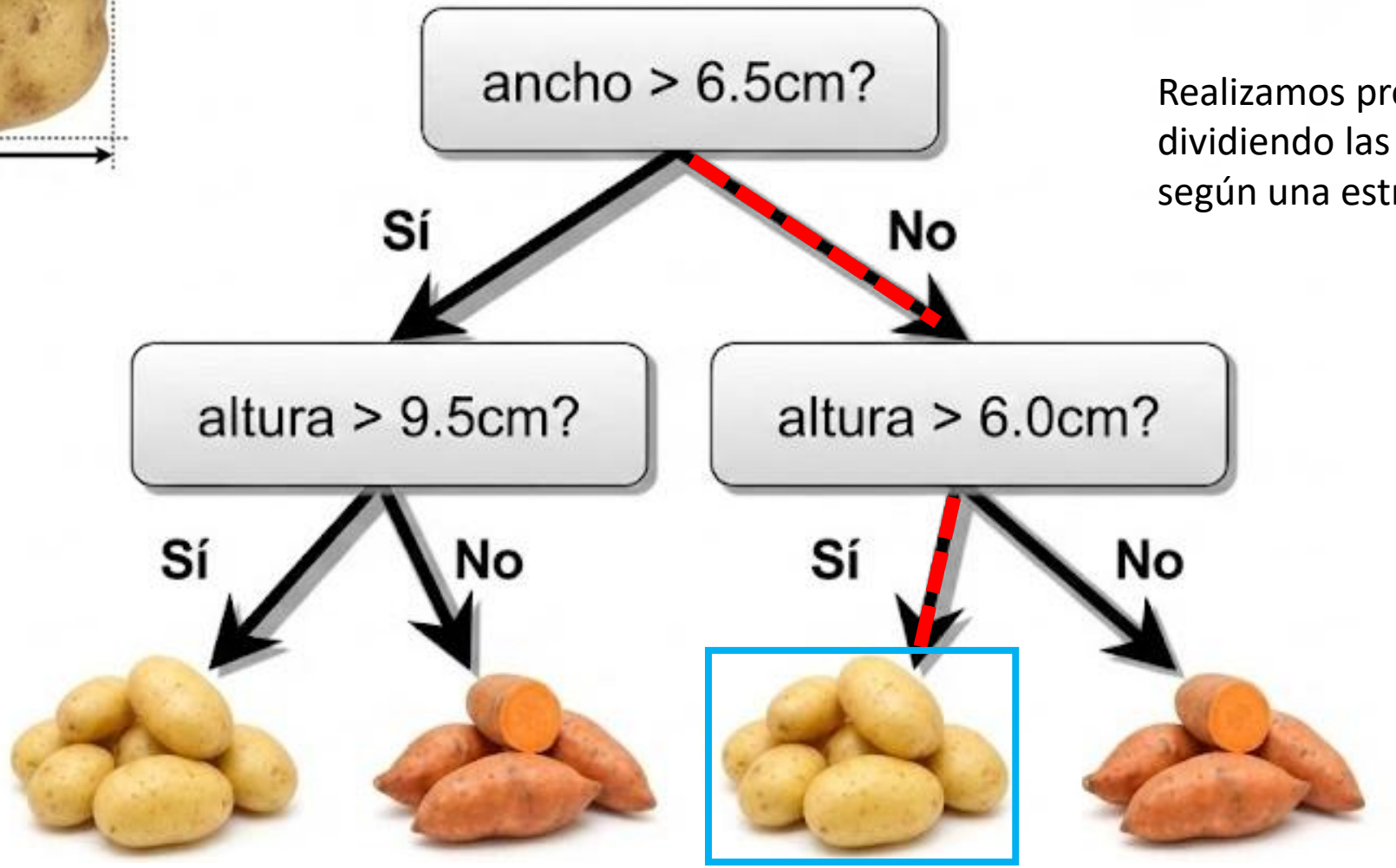
Árboles de decisión



Realizamos predicciones dividiendo las características según una estructura de árbol.

Árboles de decisión

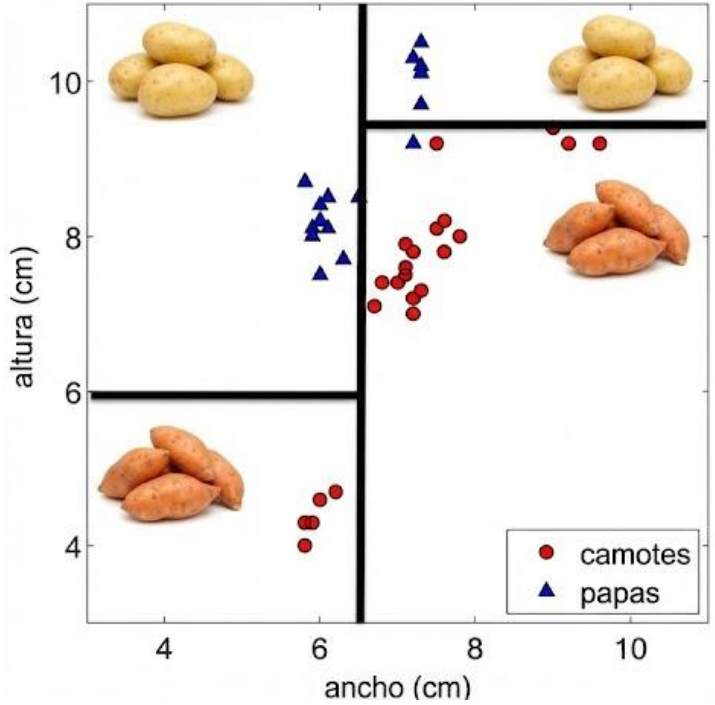


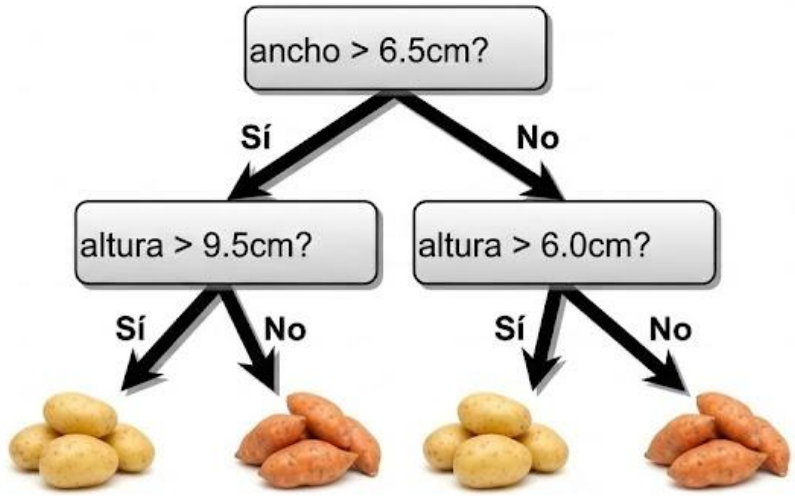


Realizamos predicciones dividiendo las características según una estructura de árbol.

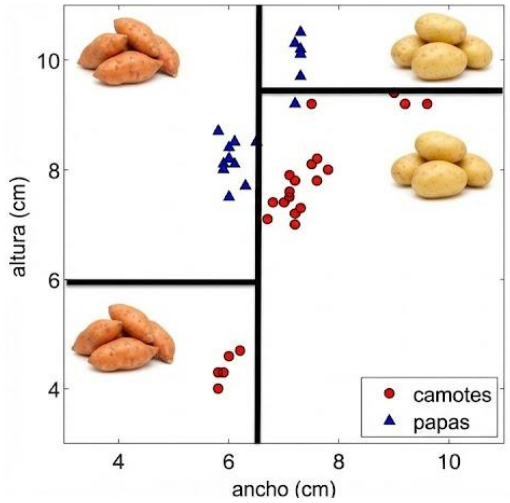
Árboles de decisión

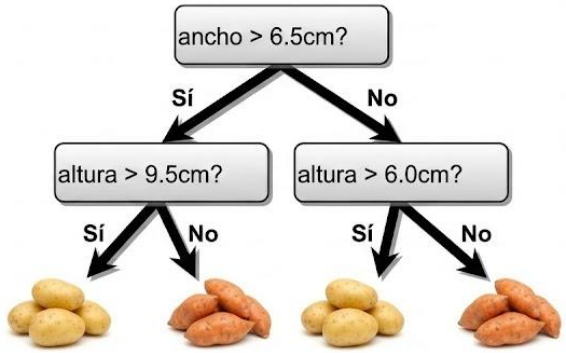
Los árboles de decisión realizan predicciones dividiéndose recursivamente en diferentes atributos según una estructura de árbol.





Construcción de árboles de decisión





En cada nivel, hay que elegir:

1. Qué variable dividir.
2. Posiblemente dónde dividirlo.

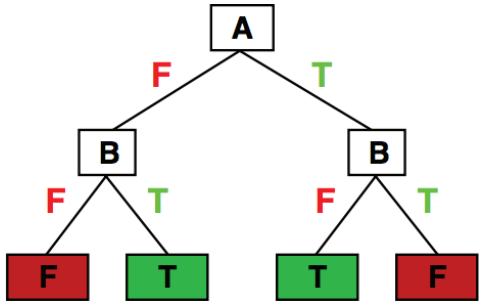
Elíjalos en función de la cantidad de información que ganaríamos con la decisión. (elija el atributo que le proporcione la mayor ganancia)

Expresividad de los árboles de decisión

Caso de entrada discreta y salida discreta:

- Los árboles de decisión pueden expresar cualquier función de los atributos de entrada.
- Por ejemplo, para las funciones booleanas, fila de la tabla de verdad → camino a la hoja:

A	B	A xor B
F	F	F
F	T	T
T	F	T
T	T	F



Caso de entrada continua y salida continua:

- Puede aproximar arbitrariamente cualquier función

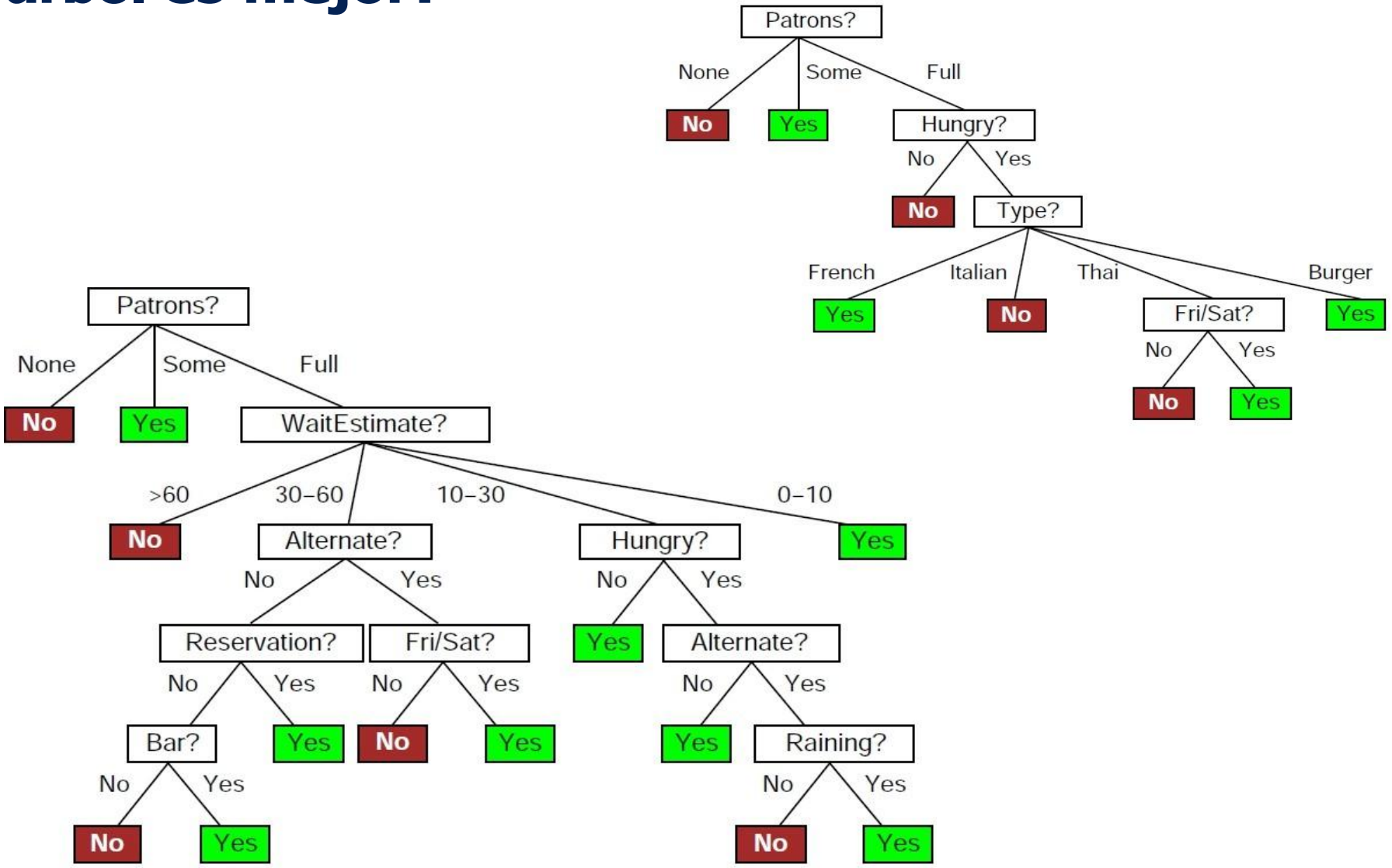
Trivialmente, existe un árbol de decisión consistente para cualquier conjunto de entrenamiento con un camino a la hoja para cada ejemplo (a menos que *f* no sea determinista en *x*) pero probablemente no se generalizará a nuevos ejemplos.

Algoritmo de construcción del árbol de decisión

Enfoque simple, codicioso y recursivo, construye el árbol nodo a nodo

1. elegir un atributo para dividir en un nodo no terminal
2. dividir los ejemplos en grupos en función del valor del atributo
3. para cada grupo:
    - **if** no hay ejemplos - devuelva la mayoría del padre
    - **else if** todos los ejemplos están en la misma clase - return class
    - else bucle al paso 1

¿Qué árbol es mejor?





¿Qué hace que un árbol sea bueno?

- No demasiado pequeño: necesidad de manejar distinciones importantes pero posiblemente sutiles en los datos
- No demasiado grande:
  - Eficiencia computacional (evitar atributos redundantes y espurios)
  - Evitar el sobreajuste de los ejemplos de entrenamiento
  - Interpretabilidad humana
- Deseamos árboles pequeños con nodos informativos cerca de la raíz



¿Cómo se entrena un árbol de decisión?



Aprendizaje de árboles de decisión

Aprender el árbol de decisión más simple (más pequeño) es un problema NP completo [si le interesa, consulte: Hyafil & Rivest'76].

- Recurrir a una heurística codiciosa:
  - Partir de un árbol de decisión vacío
  - Dividirse en el "mejor" atributo
  - Recurse
- ¿Qué atributo es el "**mejor**"?
  - ¿Elegir en función de la precisión?

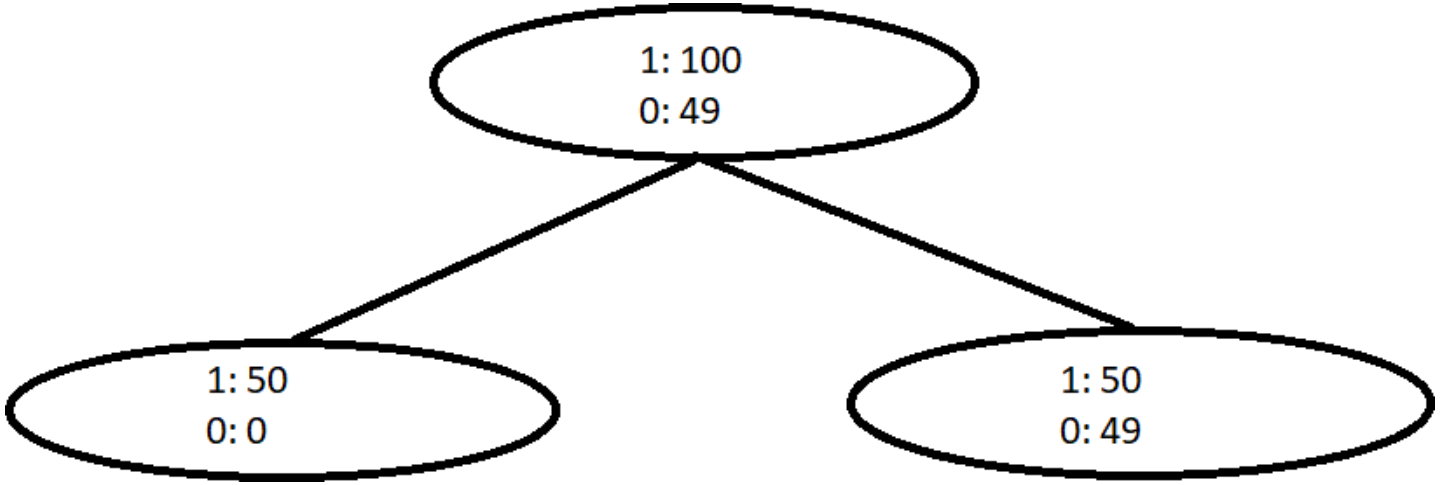
¿Se podrá realizar con descenso de gradiente?

¿Qué función de perdida elegiríamos?



Elegir una buena división

¿Por qué la precisión no es una buena medida?



¿Es buena esta división? Cero ganancia de precisión.

En su lugar, utilizaremos técnicas de la teoría de la información

Idea: Utilizar recuentos en las hojas para definir distribuciones de probabilidad, de modo que podamos medir la incertidumbre.

¿Cómo elegir una buena división?

¿Qué atributo es mejor dividir, X_1 o X_2 ?

Determinista: bueno (todos son verdaderos o falsos; sólo una clase en la hoja) (baja incertidumbre)

Distribución uniforme: mala (todas las clases de la hoja son igual de probables) (alta incertidumbre)

¿Y las distribuciones intermedias?

Nota: Demos un pequeño rodeo y recordemos conceptos de la teoría de la información

Recordar que las variables las podemos considerar “Variables aleatorias”

Tiramos dos monedas diferentes

Secuencia 1:

0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 ... ?

Secuencia 2:

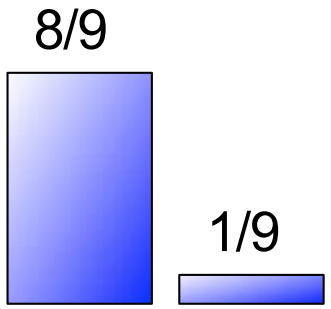
0 1 0 1 0 1 1 1 0 1 0 0 1 1 0 1 0 1 ... ?

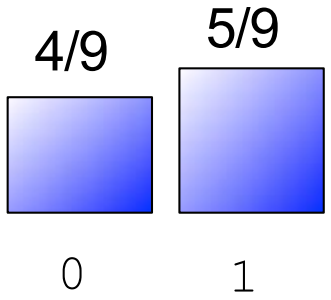


Cuantificar la incertidumbre

La entropía es una medida de la "sorpresa" esperada:

$$H(X) = -\sum_{x \in X} p(x) \log_2 p(x)$$





$$-\frac{8}{9}\log_2\frac{8}{9} - \frac{1}{9}\log_2\frac{1}{9} \approx \frac{1}{2}$$

$$-\frac{4}{9}\log_2\frac{4}{9} - \frac{5}{9}\log_2\frac{5}{9} \approx 0.99$$

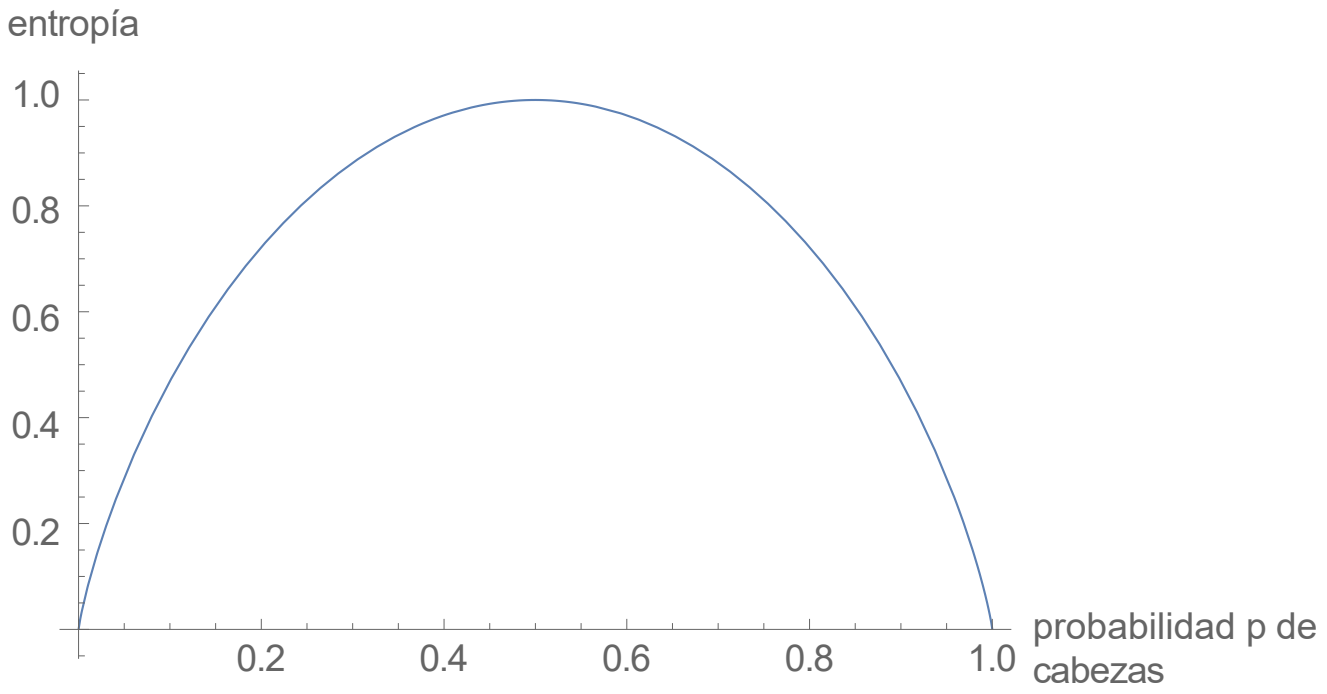
Mide el contenido informativo de cada observación

Unidad = bits

Un lanzamiento de moneda justo tiene 1 bit de entropía

Cuantificar la incertidumbre

$$H(X) = -\sum_{x \in X} p(x) \log_2 p(x)$$



Entropía

"Alta Entropía":

- La variable tiene una distribución uniforme
- Histograma plano
- Los valores muestreados de ella son menos predecibles

"Baja Entropía"

- La distribución de la variable tiene muchos picos y valles
- El histograma tiene muchos mínimos y máximos
- Los valores muestreados a partir de ella son más predecibles



Entropía condicional

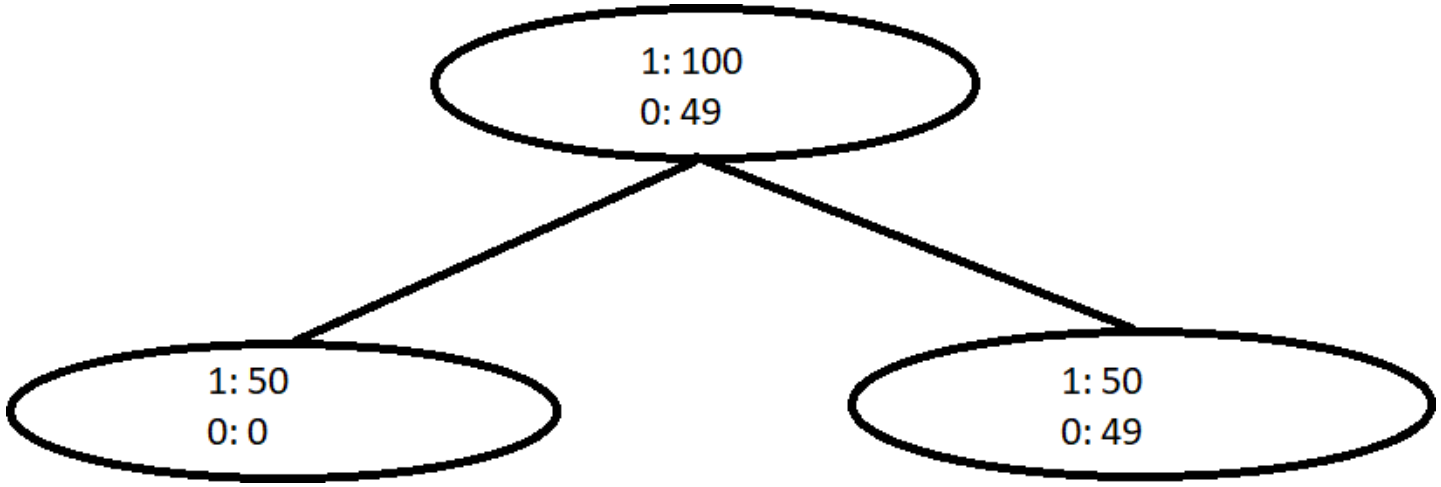
Algunas propiedades útiles:

- H es siempre no negativo
- Regla de la cadena: $H(X, Y) = H(X|Y) + H(Y) = H(Y|X) + H(X)$
- Si X e Y son independientes, entonces X no nos dice nada sobre Y:
  - $H(Y|X) = H(Y)$
- Pero Y nos dice todo sobre Y: $H(Y|Y) = 0$
- Conociendo X, sólo podemos disminuir la incertidumbre sobre Y:
  - $H(Y|X) \leq H(Y)$

Revisando nuestro ejemplo original

La ganancia de información mide la informatividad de una variable, ¡que es exactamente lo que deseamos en un atributo de un árbol de decisión!

¿Cuál es la ganancia de información de esta división?



Entropía de la raíz: $H(Y) = -\frac{49}{149}\log_2(\frac{49}{149}) - \frac{100}{149}\log_2(\frac{100}{149}) \approx 0.91$

Entropía de las hojas: $H(Y|left) = 0,\ H(Y|right) \approx 1.$

IG (*división*) ≈ $0.91 - (\frac{1}{3}\cdot 0 + \frac{2}{3}\cdot 1) \approx 0.24 > 0$



Métodos de ensamble: Visión general

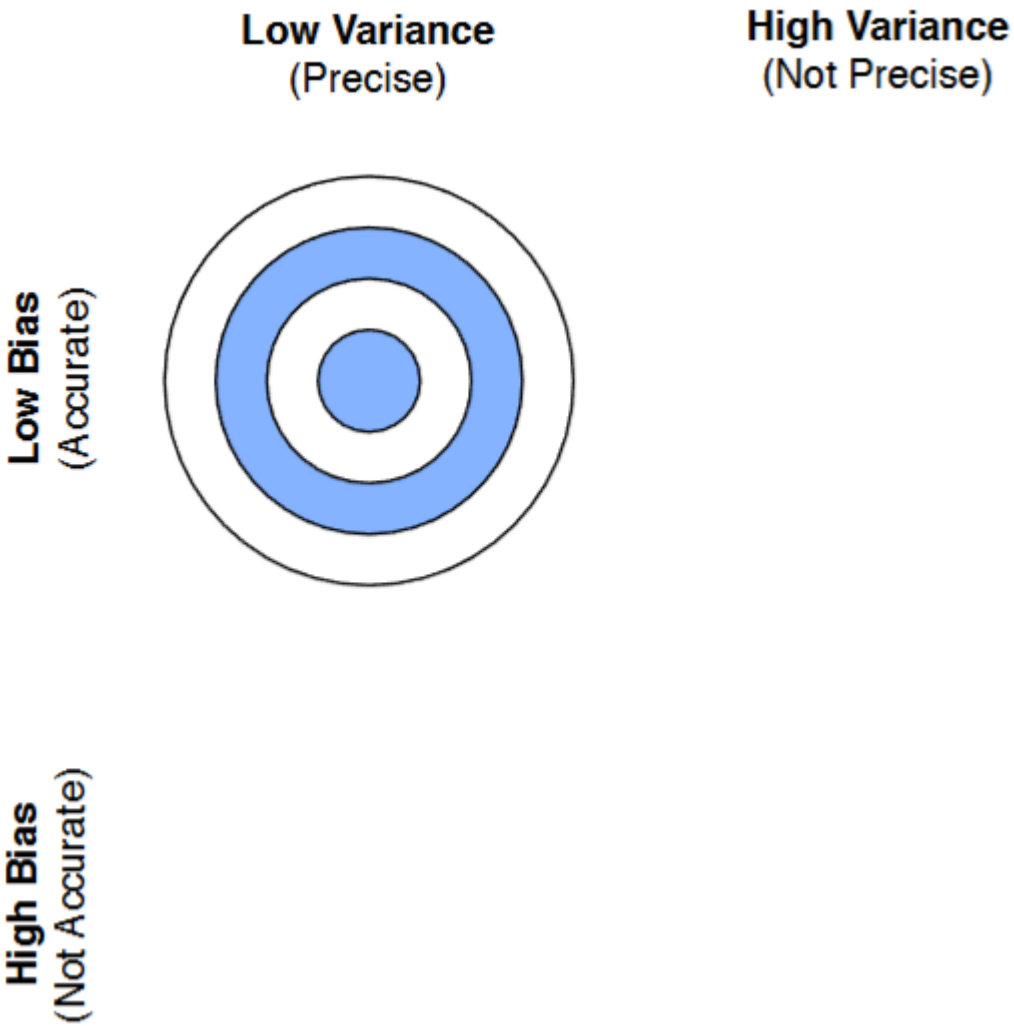
- Un ensamble de predictores es un conjunto de predictores cuyas decisiones individuales se combinan de algún modo para clasificar nuevos ejemplos
  - Por ejemplo, votación por mayoría (posiblemente ponderada)
- Para que esto no sea trivial, los clasificadores deben diferir de alguna manera, por ejemplo.
  - Algoritmo diferente
  - Diferente elección de los hiperparámetros
  - Entrenados con datos diferentes
  - Entrenado con diferente ponderación de los ejemplos de entrenamiento
- Los conjuntos suelen ser triviales de implementar. Lo difícil es decidir qué tipo de conjunto sería mejor implementar, en función de sus objetivos.



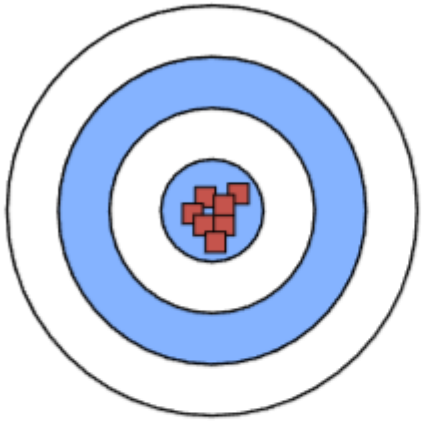
Métodos de ensamble: Visión general

- Esta conferencia: embolsado (bagging)
  - Entrenar clasificadores de forma independiente en subconjuntos aleatorios de los datos de entrenamiento.
- Conferencia posterior: impulsar (boosting)
  - Entrenar clasificadores secuencialmente, centrándose cada vez en los ejemplos de entrenamiento en los que los anteriores se equivocaron.
- El bagging y el boosting tienen propósitos muy diferentes. Para entender esto, necesitamos dar un rodeo para comprender el sesgo y la varianza de un algoritmo de aprendizaje.

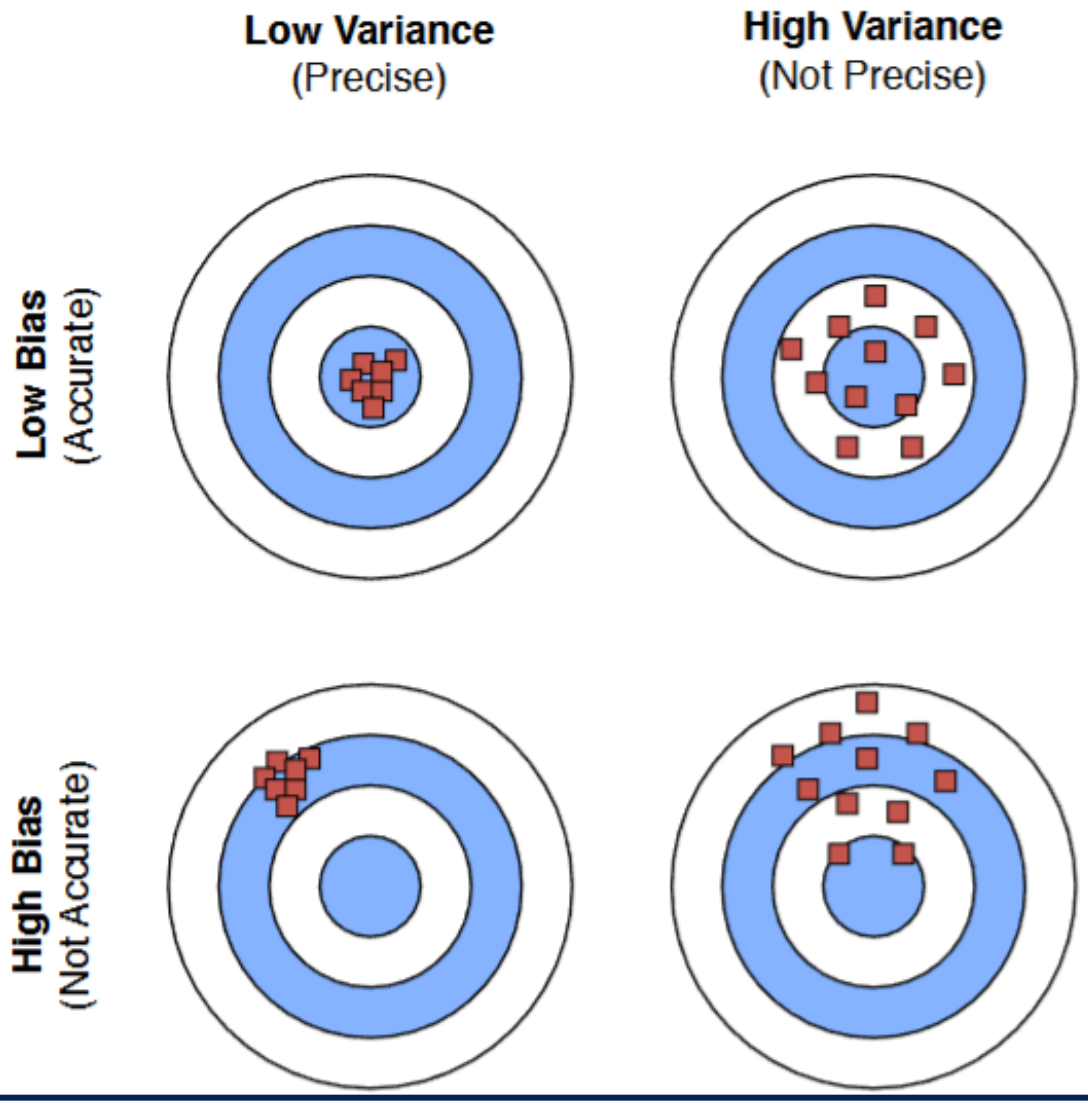
Sesgo y varianza (intuición)



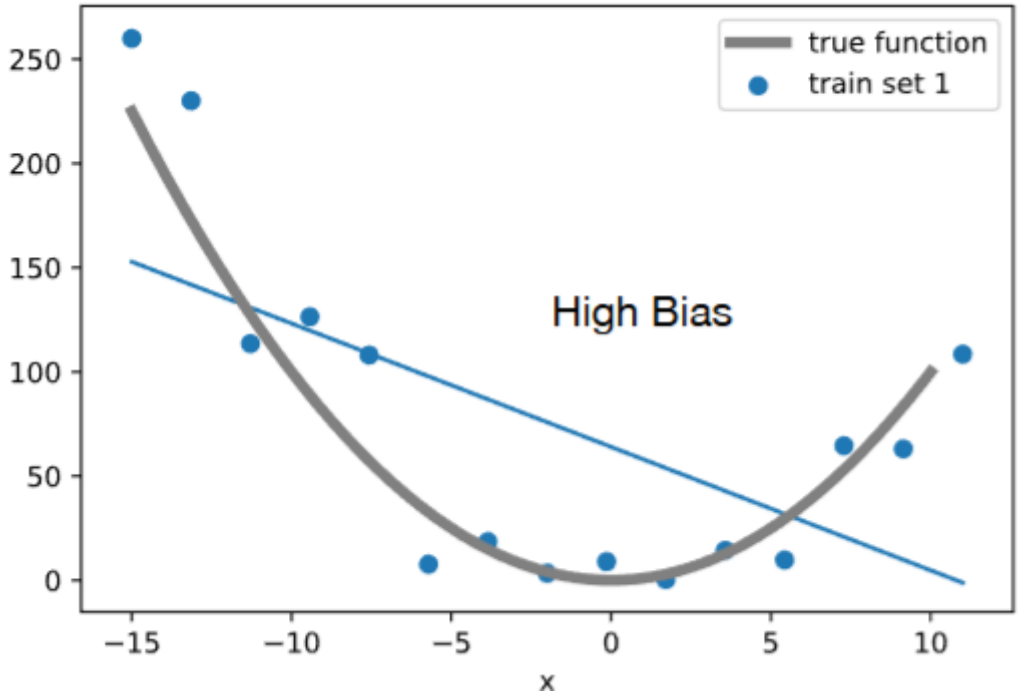
Sesgo y varianza (intuición)

	Low Variance (Precise)	**High Variance** (Not Precise)
Low Bias (Accurate)		??
High Bias (Not Accurate)	??	??

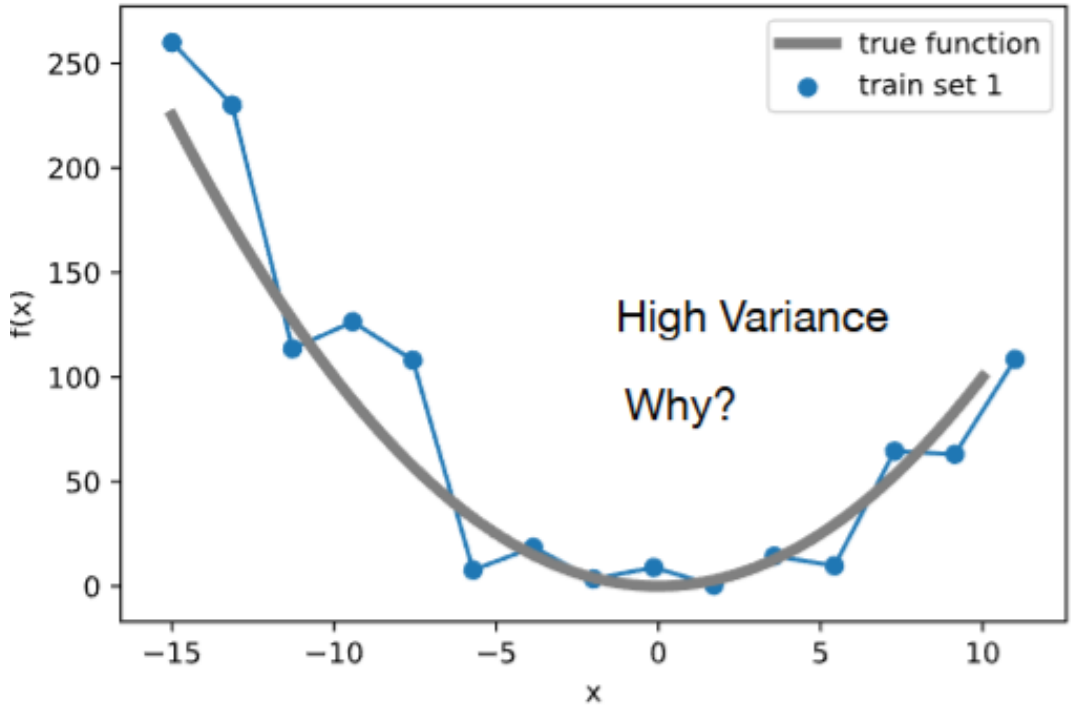
Sesgo y varianza (intuición)



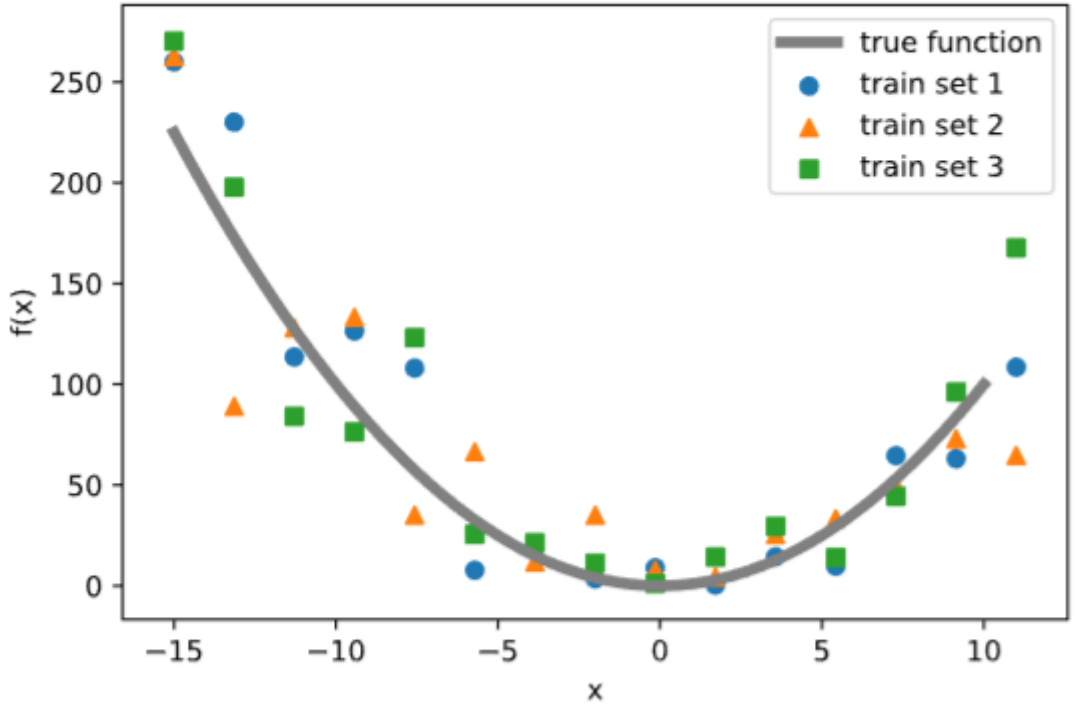
Sesgo y varianza (ejemplo)



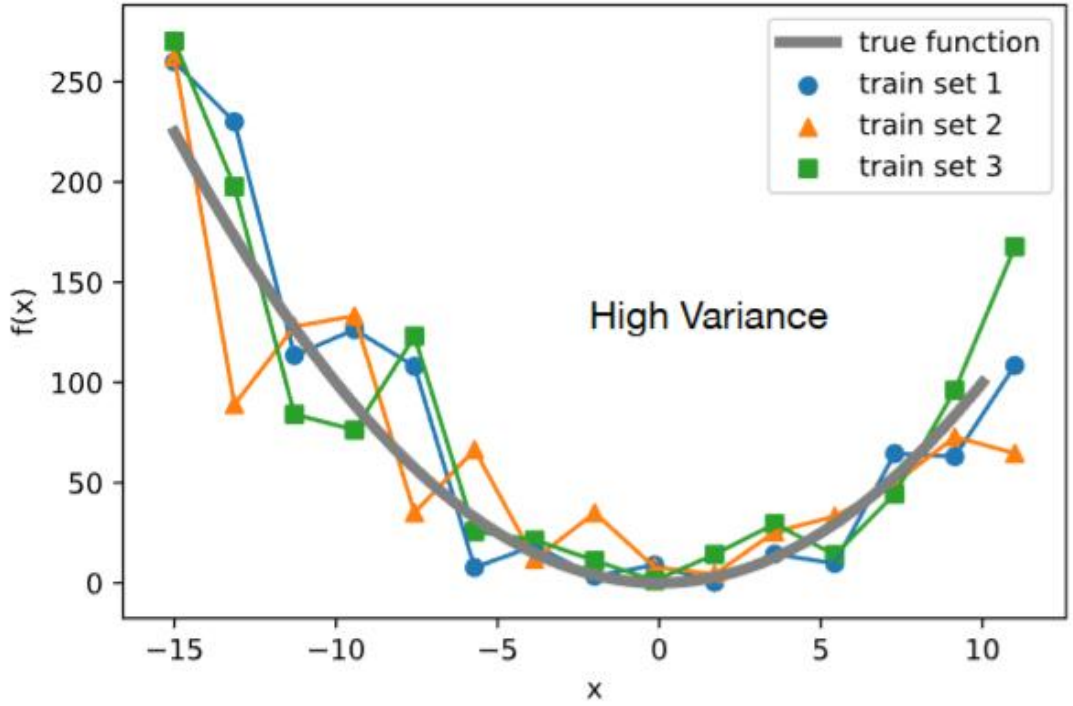
Sesgo y varianza (ejemplo)



Sesgo y varianza (ejemplo)



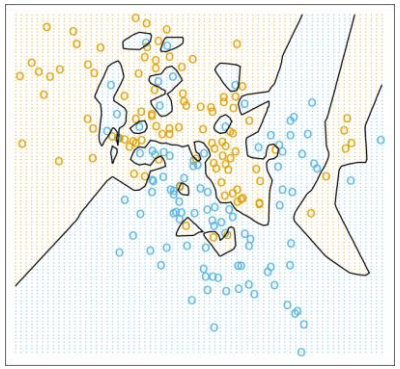
Sesgo y varianza (ejemplo)

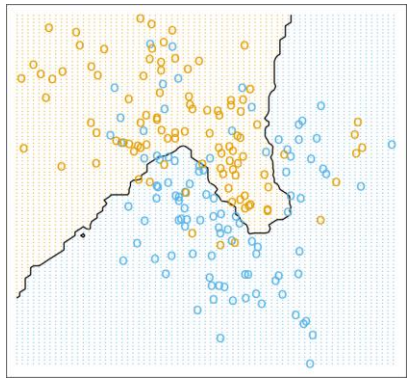


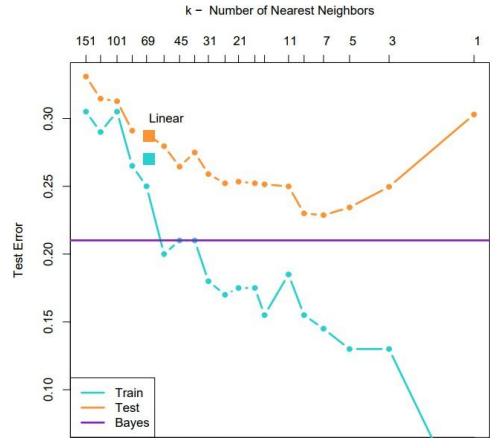
Descomposición sesgo-varianza

Recordemos que los modelos demasiado sencillos se ajustan mal a los datos,
y los modelos demasiado complejos se ajustan demasiado.

- Podemos cuantificar este efecto en términos de la descomposición sesgo/varianza.
  - ¿Sesgo y varianza de qué?





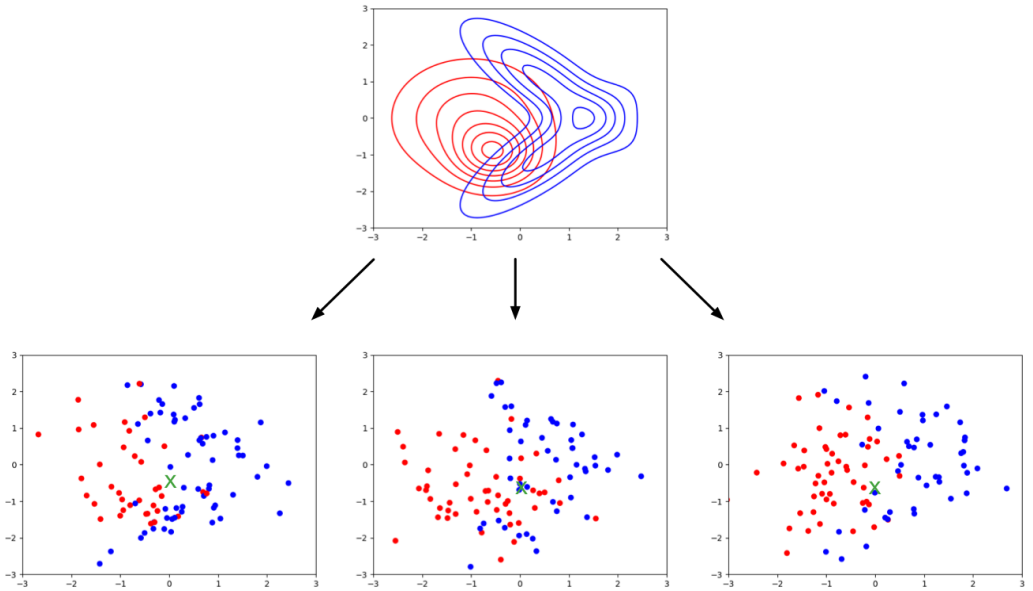


Descomposición sesgo-varianza

Supongamos que el conjunto de entrenamiento D consiste en pares $(\mathbf{x}_i\ ,\ t_i\ )$ muestreados independientes e idénticamente distribuidos (i.i.d.) a partir de una única distribución generadora de p_{data}.

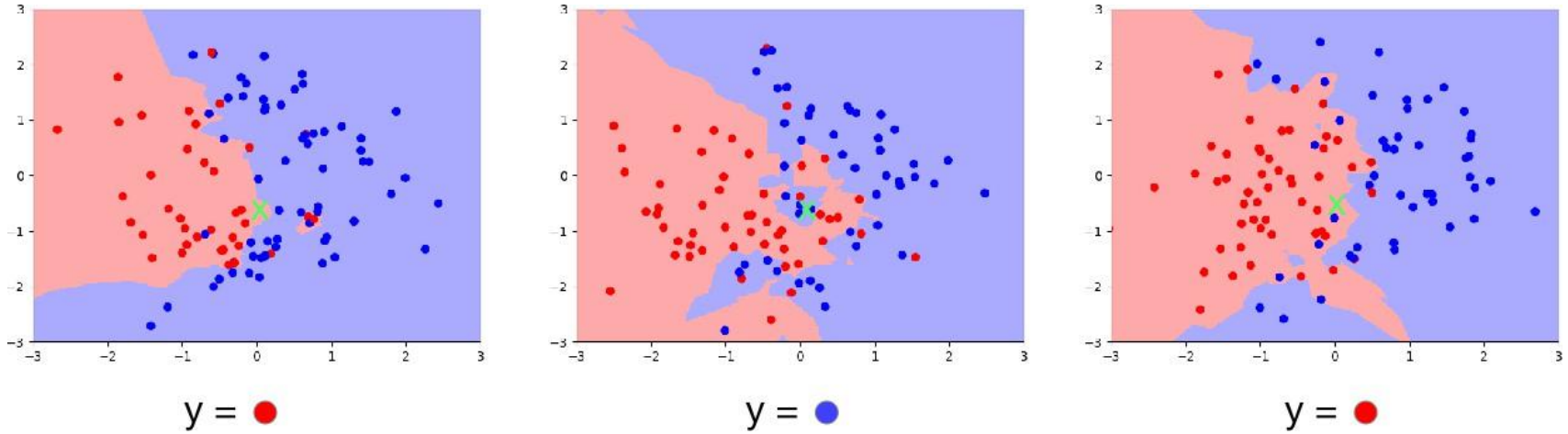
Elija un punto de consulta fijo **x** (denotado con una x verde).

Consideremos un experimento en el que muestreamos lotes de conjuntos de entrenamiento independientemente de p_{data}



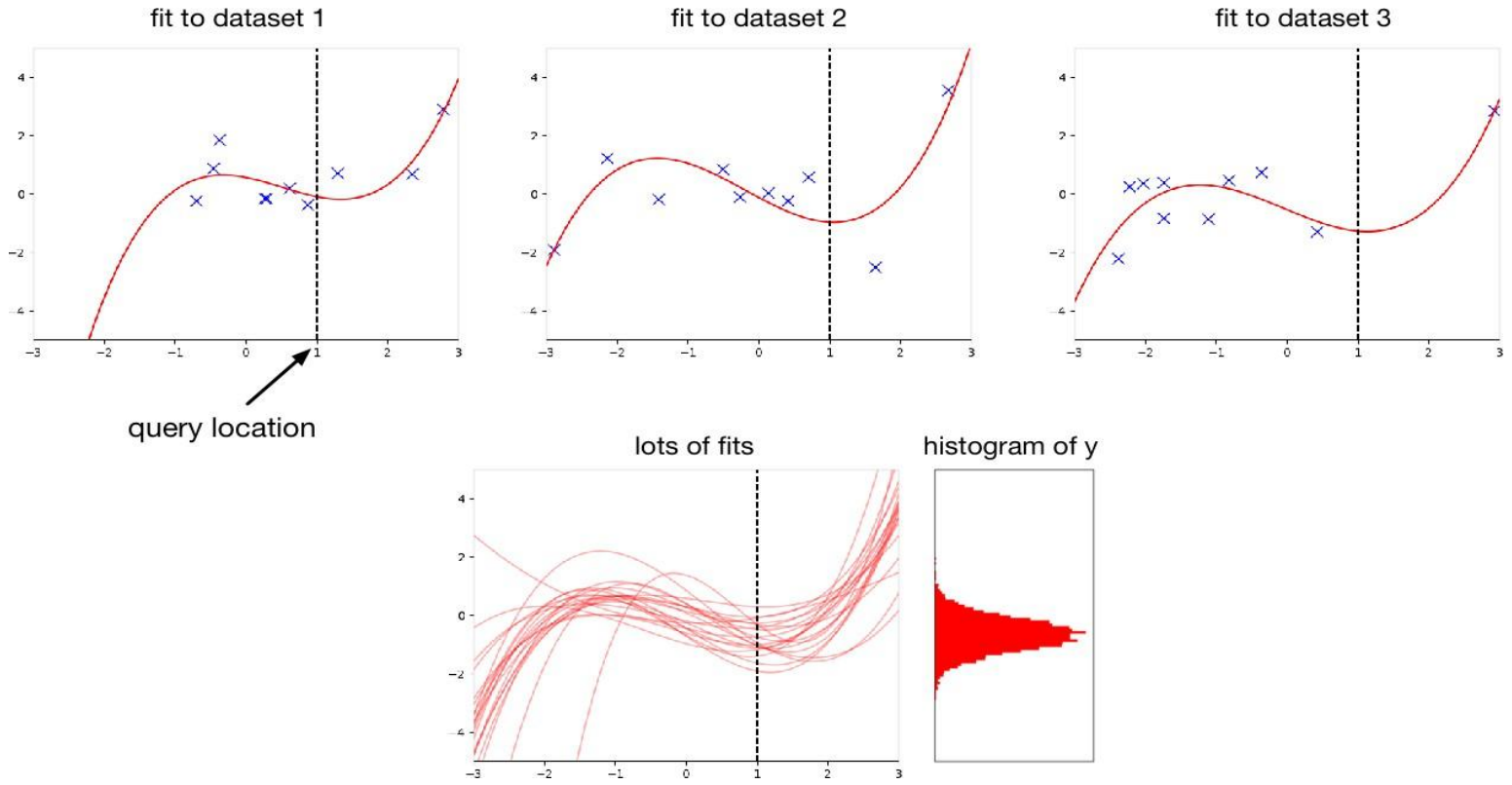
Descomposición Sesgo-Varianza

- Ejecutemos nuestro algoritmo de aprendizaje en cada conjunto de entrenamiento y calculemos su predicción *y* en el punto de consulta **x**.
- Podemos considerar *y* como una variable aleatoria, en la que la aleatoriedad procede de la elección del conjunto de entrenamiento.
- La precisión de la clasificación viene determinada por la distribución de *y* .



Descomposición Sesgo-Varianza

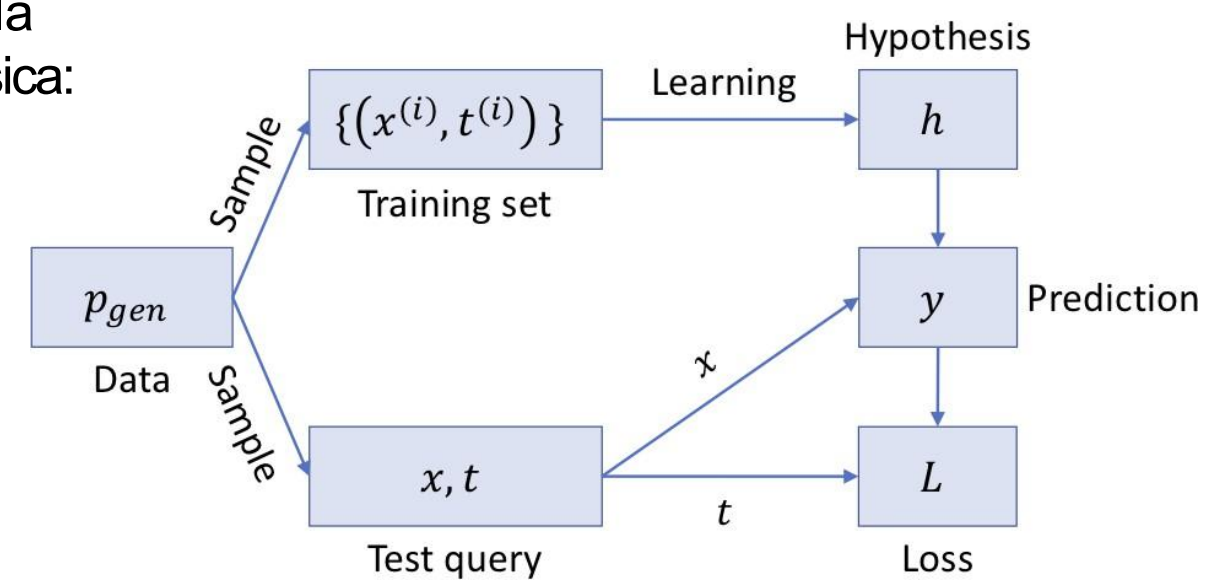
He aquí la configuración análoga para la regresión:



Dado que *y* es una variable aleatoria, podemos hablar de su expectativa, varianza, etc.

Descomposición Sesgo-Varianza

Recapitulación de la configuración básica:



Fíjese: *y* es independiente de *t*. (¿Por qué?)

Esto da una distribución sobre la pérdida en **x**, con la expectativa E[$L(y, t) \mid \mathbf{x}$].

Para cada punto de consulta **x**, la pérdida esperada es diferente. Nos interesa minimizar la expectativa de ésta con respecto a $\mathbf{x} \sim p_{\text{data}}$.



Óptimo probabilístico

Por ahora, centrémonos en la pérdida por error al cuadrado $L(y,t) = \frac{1}{2}(y-t)^2$.

Un primer paso: supongamos que conocemos la distribución condicional $p(t \mid \mathbf{x})$. ¿Qué valor y deberíamos predecir?

- En este caso, tratamos t como una variable aleatoria y elegimos y .

Afirmación: y_* = E[$t \mid \mathbf{x}$] es la mejor predicción posible.

Prueba:

$$\begin{aligned}
\mathbb{E}[(y-t)^2 \mid \mathbf{x}] &= \mathbb{E}[y^2 - 2yt + t^2 \mid \mathbf{x}] \\
&= y^2 - 2y\mathbb{E}[t \mid \mathbf{x}] + \mathbb{E}[t^2 \mid \mathbf{x}] \\
&= y^2 - 2y\mathbb{E}[t \mid \mathbf{x}] + \mathbb{E}[t \mid \mathbf{x}]^2 + \mathrm{Var}[t \mid \mathbf{x}] \\
&= y^2 - 2yy_* + y_*^2 + \mathrm{Var}[t \mid \mathbf{x}] \\
&= (y-y_*)^2 + \mathrm{Var}[t \mid \mathbf{x}]
\end{aligned}$$



Óptimo probabilístico

$$\mathbb{E}[(y - t)^2 \,|\, \mathbf{x}] = (y - y_*)^2 + \mathrm{Var}[t \,|\, \mathbf{x}]$$

El primer término es no negativo, y puede hacerse 0 fijando $y = y_*$

El segundo término corresponde a la imprevisibilidad inherente, o ruido, de los objetivos, y se denomina error de Bayes.

- Esto es lo mejor que podemos esperar de cualquier algoritmo de aprendizaje.
  - Un algoritmo que lo consiga es el óptimo de Bayes o probabilístico.
- Observe que este término no depende de y .

Este proceso de elección de un único valor y_* basado en $p(t \,|\, \mathbf{x})$ es un ejemplo de teoría de la decisión.

Óptimo probabilístico

Ahora volvamos a tratar *y* como una variable aleatoria (donde la aleatoriedad proviene de la elección del conjunto de datos).

Podemos descomponer la pérdida esperada (suprimiendo el condicionamiento sobre **x**
para mayor claridad):

$$
\begin{aligned}
\mathbb{E}[(y-t)^2] &= \mathbb{E}[(y-y_*)^2] + \mathrm{Var}(t) \\
&= \mathbb{E}[y_*^2 - 2y_*y + y^2] + \mathrm{Var}(t) \\
&= y_*^2 - 2y_*\mathbb{E}[y] + \mathbb{E}[y^2] + \mathrm{Var}(t) \\
&= y_*^2 - 2y_*\mathbb{E}[y] + \mathbb{E}[y]^2 + \mathrm{Var}(y) + \mathrm{Var}(t) \\
&= \underbrace{(y_* - \mathbb{E}[y])^2}_{\text{bias}} + \underbrace{\mathrm{Var}(y)}_{\text{variance}} + \underbrace{\mathrm{Var}(t)}_{\text{Bayes error}}
\end{aligned}
$$



Descomposición sesgo/varianza

Recordemos que tratamos las predicciones y en una consulta **x** como una variable aleatoria (donde la aleatoriedad procede de la elección del conjunto de datos), y_* es la predicción determinista óptima, t es un objetivo aleatorio muestreado a partir del condicional verdadero p(**t|x**).

$$\mathbb{E}[(y-t)^2] = \underbrace{(y_\star - \mathbb{E}[y])^2}_{\text{bias}} + \underbrace{\text{Var}(y)}_{\text{variance}} + \underbrace{\text{Var}(t)}_{\text{Bayes error}}$$

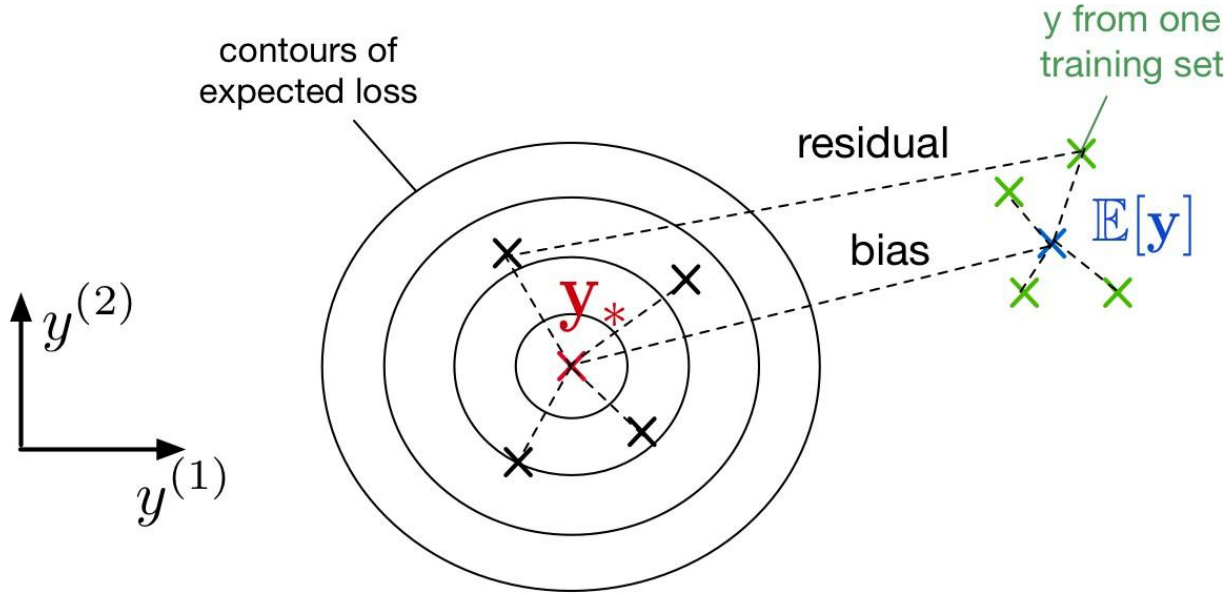
El sesgo/varianza descompone la pérdida esperada en tres términos:

- sesgo: lo errónea que es la predicción esperada (corresponde a un ajuste insuficiente)
- varianza: la cantidad de variabilidad en las predicciones (corresponde al sobreajuste)
- error de bayes: la imprevisibilidad inherente de los objetivos

Aunque este análisis sólo se aplica al error al cuadrado, a menudo utilizamos vagamente "sesgo" y "varianza" como sinónimos de "inadaptación" y "sobreadaptación".

Descomposición sesgo/varianza

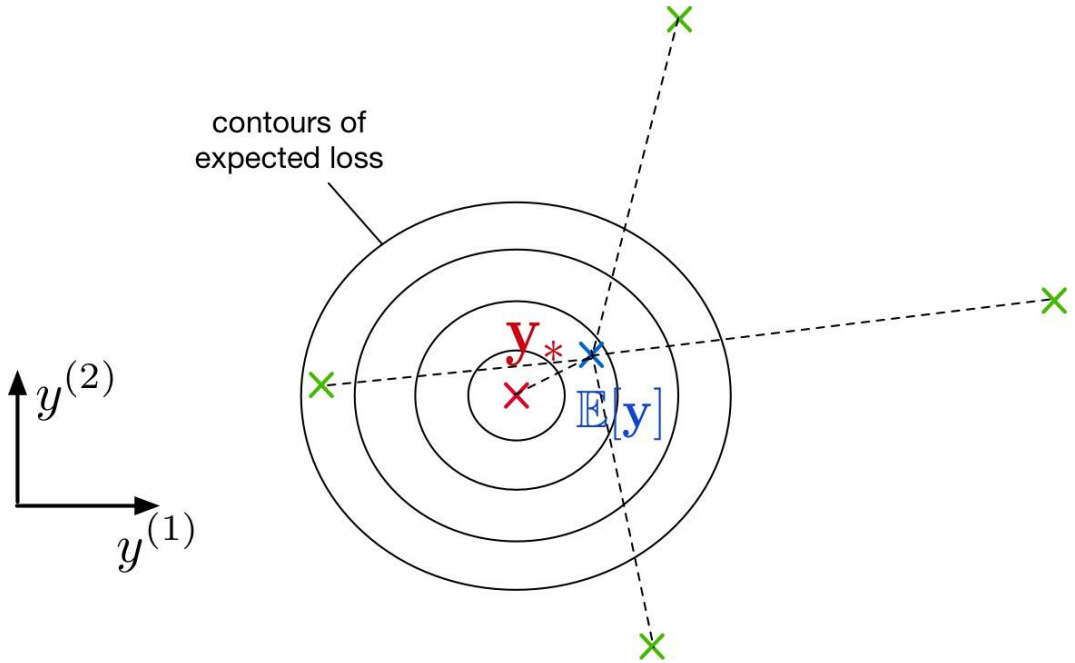
- Podemos visualizar esta descomposición en el espacio de salida, donde los ejes corresponden a las predicciones sobre los ejemplos de prueba.
- Si tenemos un modelo demasiado simple (por ejemplo, K N N con $\mathbf{k}$ grande), podría tener
  - alto sesgo (porque no puede captar la estructura de los datos)
  - baja varianza (porque hay suficientes datos para obtener estimaciones estables)



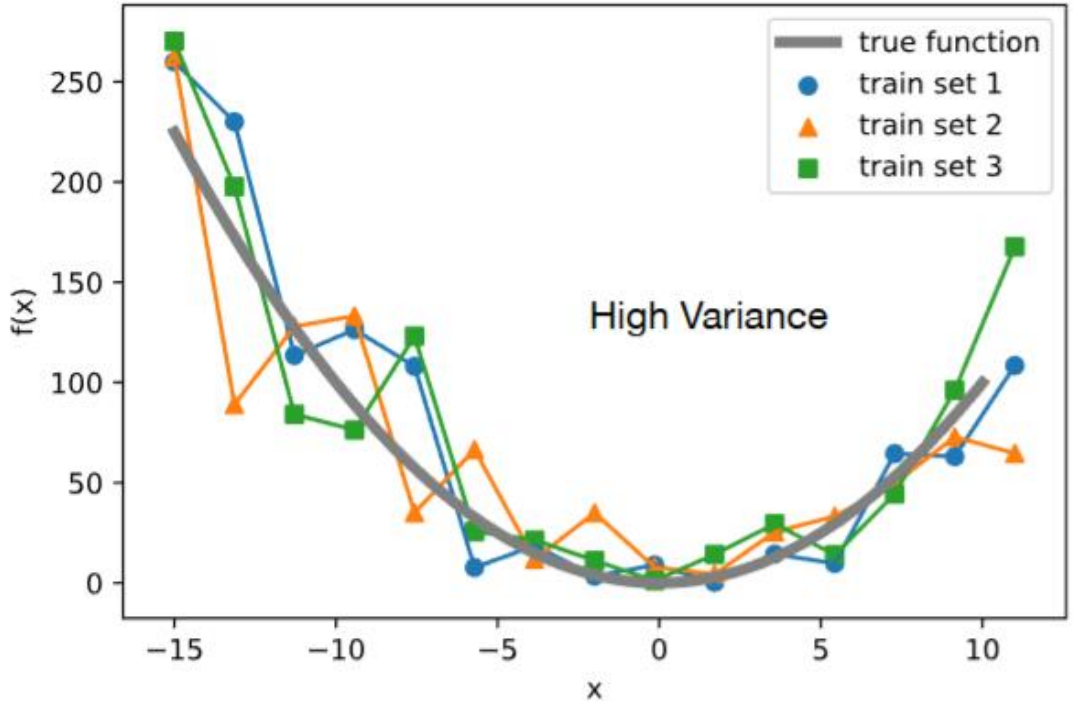
Descomposición sesgo/varianza

Si tiene un modelo demasiado complejo (por ejemplo, K N N con $k = 1$), podría tener

- bajo sesgo (ya que aprende toda la estructura relevante)
- alta varianza (se ajusta a las peculiaridades de los datos que ha muestreado por casualidad)



¿Qué hará el bagging?



(10 min) Break



Bagging: Motivación

Supongamos que pudiéramos muestrear de algún modo m conjuntos de entrenamiento independientes de P_{sample}.

A continuación, podríamos calcular la predicción y_i basándonos en cada una de ellas, y tomar la media

$$y = \frac{1}{m}\sum_{i=1}^{m} y_i$$

¿Cómo afecta esto a los tres términos de la pérdida esperada?

Error de Bayes: sin cambios, ya que no tenemos control sobre él

Sesgo: sin cambios, ya que la predicción promediada tiene la misma expectativa

$$\mathbb{E}[y] = \mathbb{E}\left[\frac{1}{m}\sum_{i=1}^{m} y_i\right] = \mathbb{E}[y_i]$$

Varianza: reducida, ya que estamos promediando sobre muestras independientes

$$\mathrm{Var}[y] = \mathrm{Var}\left[\frac{1}{m}\sum_{i=1}^{m} y_i\right] = \frac{1}{m^2}\sum_{i=1}^{m}\mathrm{Var}[y_i] = \frac{1}{m}\mathrm{Var}[y_i].$$



Bagging: La idea

En la práctica, la distribución de muestreo p_{sample} suele ser finita o costosa de muestrear.

Así pues, entrenar modelos separados en conjuntos de datos muestreados de forma independiente supone un gran derroche de datos.

¿Por qué no entrenar un único modelo en la unión de todos los conjuntos de datos muestreados?

Solución: dado el conjunto de entrenamiento D, utilice la distribución empírica p_D como sustituto de p_{sample}. Esto se denomina agregación bootstrap, o bagging .

Tome un único conjunto de datos D con n ejemplos.

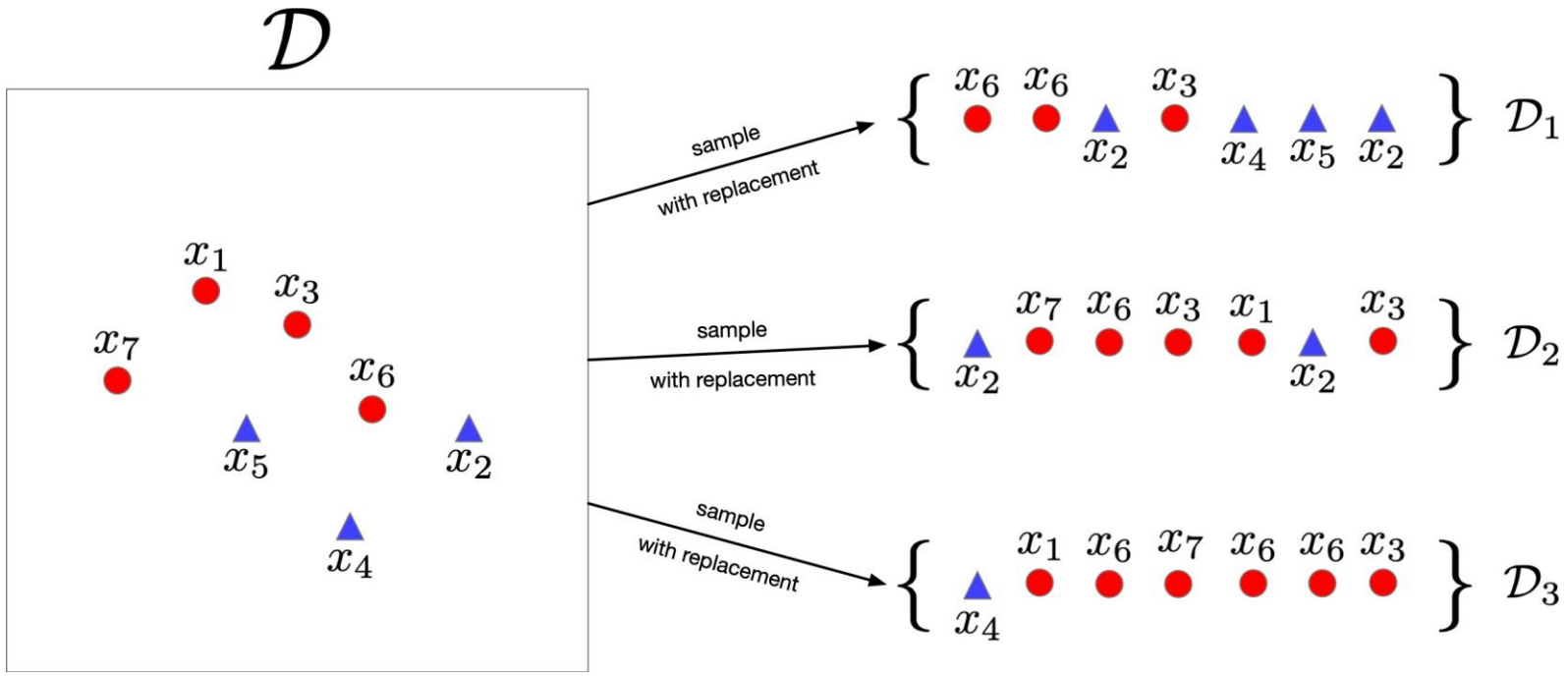
Genere m nuevos conjuntos de datos ("remuestreos" o " muestras bootstrap"), cada uno mediante el muestreo de n ejemplos de entrenamiento de D, con reemplazo.

Promedie las predicciones de los modelos entrenados en cada uno de estos conjuntos de datos.

El bootstrap es una de las ideas más importantes de toda la estadística.

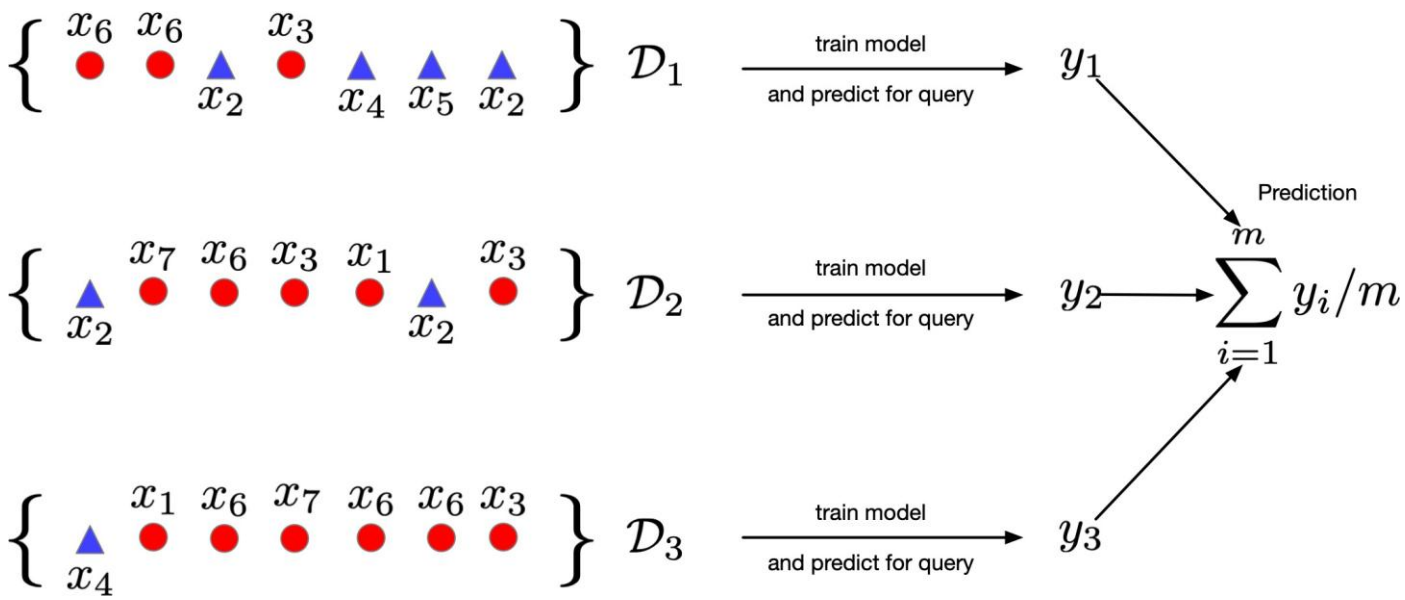
Intuición: Como $|D| \to \infty$, tenemos $p_D \to P_{sample}$.

Bagging



en este ejemplo n = 7, m = 3

Bagging



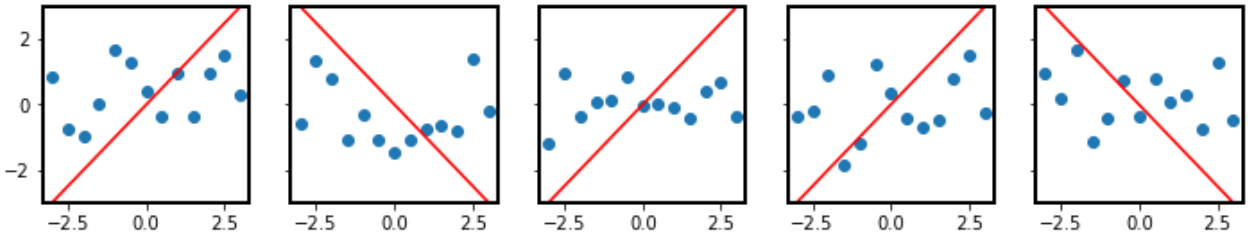
predicción sobre un punto de consulta x

Bagging: Efecto sobre el espacio de hipótesis

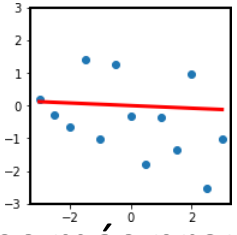
Ejemplo ilustrativo:

$$x \sim U(-3, 3), t \sim N(0, 1)$$
$$H = wx \mid w \in \{-1, 1\}$$

Conjuntos de datos muestreados e hipótesis ajustadas:



Hipótesis ensambladas (media sobre 1000 muestras):



¡La hipótesis ensamblada no se encuentra en el espacio de hipótesis original!

Este efecto es más pronunciado cuando se combinan clasificadores …

Bagging: Efecto de la correlación

Problema: los conjuntos de datos no son independientes, por lo que no obtenemos la reducción de varianza $1/\mathrm{m}$.

- Es posible demostrar que si las predicciones muestreadas tienen una varianza σ^2 y correlación ρ, entonces

$$\mathrm{Var}\left(\frac{1}{m}\sum_{i=1}^{m} y_i\right) = \frac{1}{m}(1-\rho)\sigma^2 + \rho\sigma^2.$$

Irónicamente, puede ser ventajoso introducir *una* variabilidad *adicional* en su algoritmo, siempre que reduzca la correlación entre las muestras.

- Intuición: usted quiere invertir en una cartera diversificada, no en una sola acción.
- Puede ayudar a utilizar el promedio sobre múltiples algoritmos, o múltiples configuraciones del mismo algoritmo.



Bosques aleatorios (Random Forest)

Bosques aleatorios = árboles de decisión bagged

Al elegir cada nodo del árbol de decisión, elija un conjunto aleatorio de d características de entrada, y sólo considere las divisiones en esas características

Los bosques aleatorios son probablemente el mejor algoritmo de aprendizaje automático de caja negra: suelen funcionar bien sin ningún tipo de ajuste.

uno de los algoritmos más utilizados en las competiciones Kaggle



https://www.youtube.com/watch?v=3pa0vtW64Ic

Resumen de bagging

- El bagging reduce el sobreajuste al promediar las predicciones.
- Limitaciones:
  - No reduce el sesgo en caso de error al cuadrado.
  - Sigue habiendo correlación entre los clasificadores.
  - Mezcla ingenua (todos los miembros se ponderan por igual).
    - Si los miembros son muy diferentes (por ejemplo, diferentes algoritmos, diferentes fuentes de datos, etc.), a menudo podemos obtener mejores resultados utilizando un enfoque basado en principios para el ensamblaje ponderado.
- Boosting puede verse como un enfoque del ensamblaje ponderado que descorrelaciona fuertemente a los miembros del conjunto.

Impulsar (Boosting)

- Boosting
  - Entrenar los clasificadores secuencialmente, centrándose cada vez en los ejemplos de entrenamiento en los que los anteriores se equivocaron.
  - El cambio de enfoque descorrelaciona fuertemente sus predicciones.
- Para centrarse en ejemplos concretos, la potenciación utiliza un conjunto de entrenamiento ponderado.

Conjunto de entrenamiento ponderado

El ejemplo de tasa de clasificación $\frac{1}{N}\sum_{n=1}^{N}\mathbb{I}[h(x^{(n)}) \neq t^{(n)}]$ pondera cada entrenamiento errónea por igual.

Idea clave: podemos aprender un clasificador utilizando diferentes costes (también conocidos como pesos) para los ejemplos.

El clasificador "se esfuerza más" en los ejemplos con

mayor coste Cambie la función de coste:

$$\sum_{n=1}^{N}\frac{1}{N}\mathbb{I}[h(x^{(n)}) \neq t^{(n)}] \quad \text{becomes} \quad \sum_{n=1}^{N} w^{(n)}\mathbb{I}[h(x^{(n)}) \neq t^{(n)}]$$

Normalmente se requiere $w^{(n)} > 0$ and $\sum_{n=1}^{N} w^{(n)} = 1$

AdaBoost (boosting adaptativo)

Ahora podemos describir el algoritmo AdaBoost.

Dado un clasificador base, los pasos clave de AdaBoost son:

1. En cada iteración, vuelva a ponderar las muestras de entrenamiento asignando pesos mayores a las muestras (es decir, a los puntos de datos) que se clasificaron incorrectamente.
2. Entrene un nuevo clasificador base basado en las muestras reponderadas.
3. Añádalo al conjunto de clasificadores con un peso adecuado.
4. Repita el proceso muchas veces.

Requisitos para el clasificador de base:

Debe minimizar el error ponderado.

El conjunto puede llegar a ser muy grande, por lo que el clasificador base debe ser rápido.

Resulta que basta con cualquier clasificador/modelo débil.

Individualmente, los modelos débiles pueden tener un sesgo elevado (underfit). Al hacer que cada clasificador se centre en los errores anteriores, AdaBoost reduce el sesgo.



Modelo/Clasificador débil

(Informal) Un modelo débil es un algoritmo de aprendizaje que emite una hipótesis (por ejemplo, un clasificador) que funciona ligeramente mejor que el azar, por ejemplo, predice la etiqueta correcta con una probabilidad de 0,51 en el caso de etiqueta binaria.

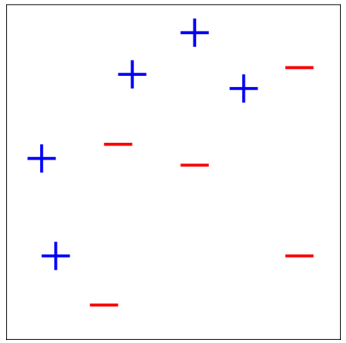
Nos interesan los aprendices débiles que sean eficientes *desde el punto de vista computacional*.

Árboles de decisión

Aún más sencillo: Árbol de decisión: Un árbol de decisión con una única división

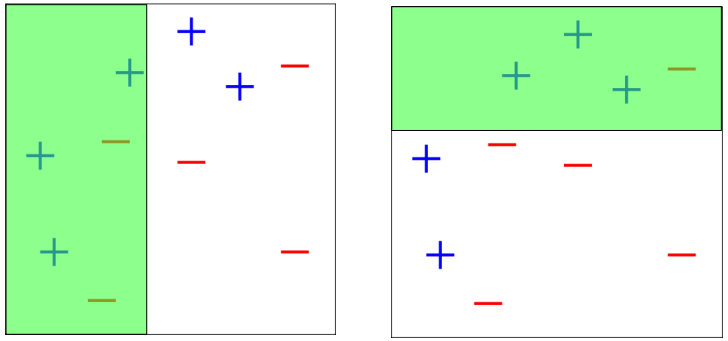
[La definición formal de la aprendibilidad débil tiene cuantificadores como "para cualquier distribución sobre los datos" y el requisito de que su garantía sólo se mantiene probabilísticamente].

Clasificadores débiles



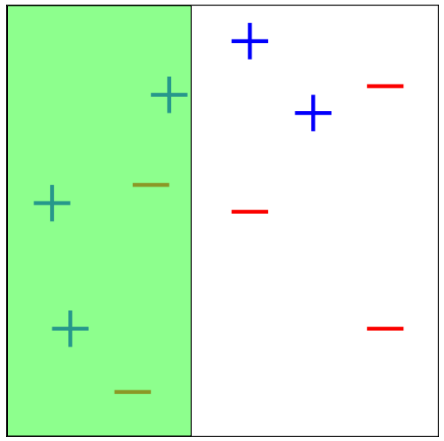
Estos clasificadores débiles, que son fronteras de decisión, consisten en el conjunto de semiespacios horizontales y verticales.

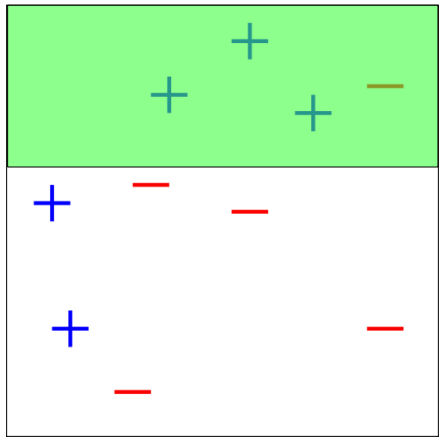
Semiespacios verticales Semiespacios horizontales



Clasificadores débiles

Semiespacios verticales





Semiespacios horizontales

Un único clasificador débil no es capaz de hacer que el error de entrenamiento sea pequeño

Pero si se puede garantizar que funciona ligeramente mejor que el azar, es decir, el error ponderado del clasificador h según las ponderaciones dadas

Veamos ahora cómo AdaBoost combina un conjunto de clasificadores débiles para formar un mejor conjunto de clasificadores...



Algoritmo AdaBoost

Input: Data $\mathcal{D}_N$, weak classifier WeakLearn (a classification procedure that returns a classifier h, e.g. best decision stump, from a set of classifiers $\mathcal{H}$, e.g. all possible decision stumps), number of iterations T

Output: Classifier $H(x)$

Initialize sample weights: $w^{(n)} = \frac{1}{N}$ for $n = 1, \ldots, N$

For $t = 1, \ldots, T$

- Fit a classifier to weighted data ($h_t \leftarrow \text{WeakLearn}(\mathcal{D}_N, \mathbf{w})$), e.g.,

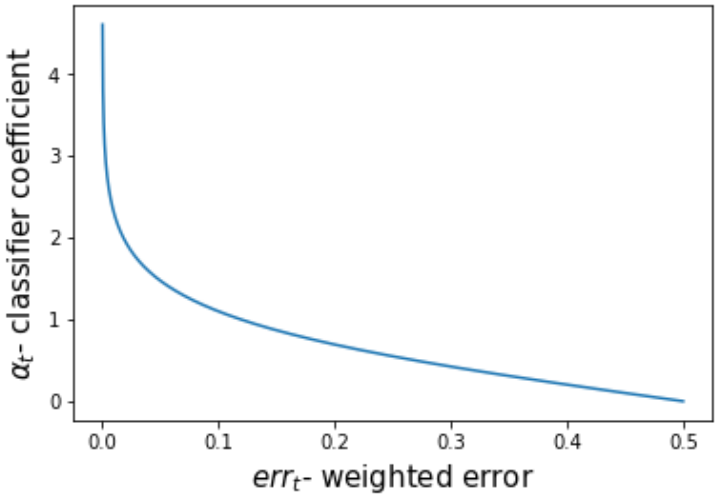
$$h_t \leftarrow \underset{h \in \mathcal{H}}{\text{argmin}} \sum_{n=1}^{N} w^{(n)} \mathbb{I}\{h(\mathbf{x}^{(n)}) \neq t^{(n)}\}$$

- Compute weighted error $\text{err}_t = \frac{\sum_{n=1}^{N} w^{(n)} \mathbb{I}\{h_t(\mathbf{x}^{(n)}) \neq t^{(n)}\}}{\sum_{n=1}^{N} w^{(n)}}$
- Compute classifier coefficient $\alpha_t = \frac{1}{2} \log \frac{1-\text{err}_t}{\text{err}_t}$ $\quad (\in (0, \infty))$
- Update data weights

$$w^{(n)} \leftarrow w^{(n)} \exp\left(-\alpha_t t^{(n)} h_t(\mathbf{x}^{(n)})\right) \left[\equiv w^{(n)} \exp\left(2\alpha_t \mathbb{I}\{h_t(\mathbf{x}^{(n)}) \neq t^{(n)}\}\right)\right]$$

Ponderar la intuición

$$H(\mathbf{x}) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(\mathbf{x})\right) \text{ where } \alpha_t = \frac{1}{2}\log\frac{1-\text{err}_t}{\text{err}_t}$$



Los clasificadores débiles que obtienen un error ponderado más bajo reciben más peso en el clasificador final

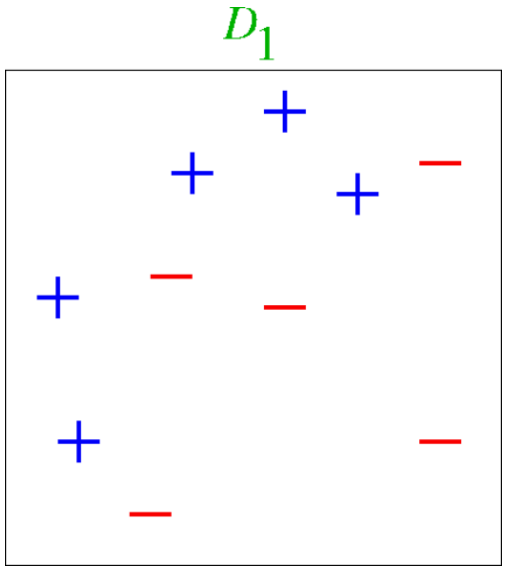
También: $w^{(n)} \leftarrow w^{(n)} \exp\left(2\alpha_t \mathbb{I}\{h_t(\mathbf{x}^{(n)}) \neq t^{(n)}\}\right)$

Si $\text{err}_t \approx 0$, α_t alto por lo que los ejemplos mal clasificados reciben más atención

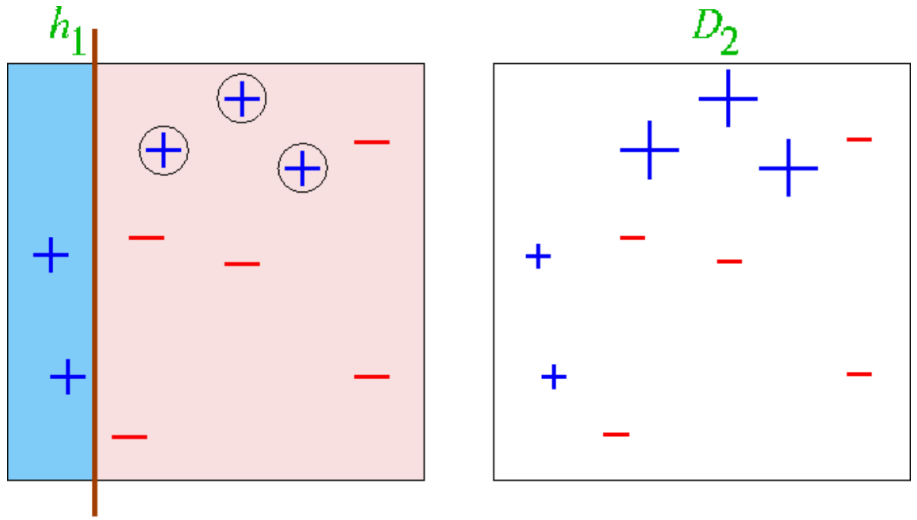
Si $\text{err}_t \approx 0{,}5$, α_t baja para que no se enfaticen los ejemplos mal clasificados.

Ejemplo de AdaBoost

Datos de entrenamiento



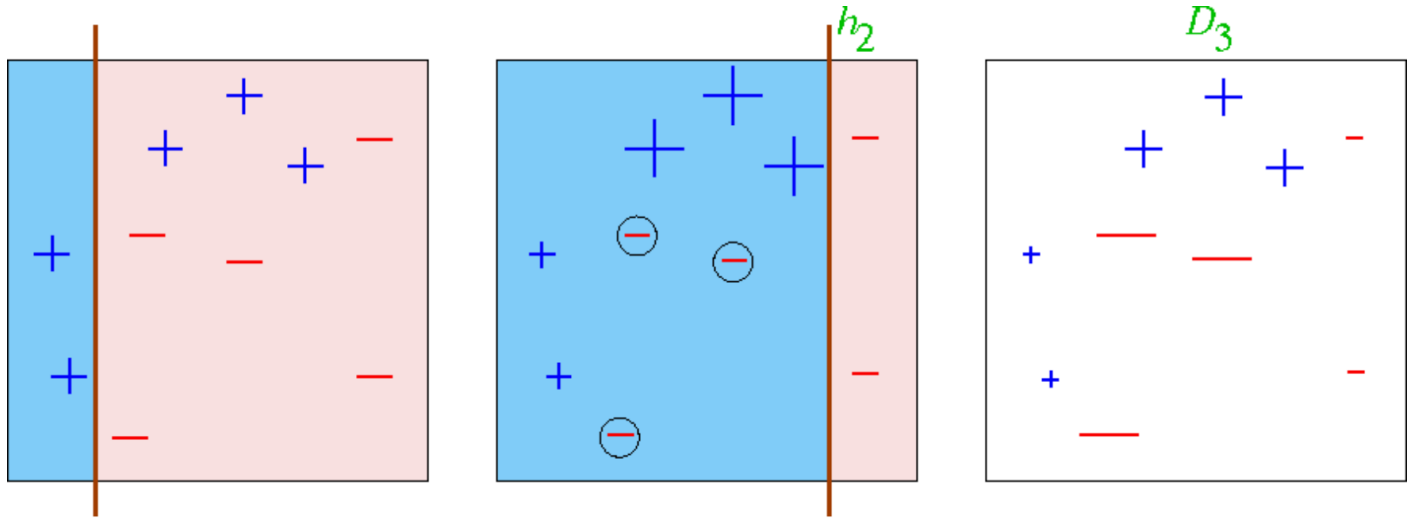
Ronda 1



$$\mathbf{w} = \left(\frac{1}{10}, \ldots, \frac{1}{10}\right) \Rightarrow \text{Train a classifier (using } \mathbf{w}) \Rightarrow \text{err}_1 = \frac{\sum_{i=1}^{10} w_i \mathbb{I}\{h_1(\mathbf{x}^{(i)}) \neq t^{(i)}\}}{\sum_{i=1}^{N} w_i} = \frac{3}{10}$$

$$\Rightarrow \alpha_1 = \frac{1}{2} \log \frac{1 - \text{err}_1}{\text{err}_1} = \frac{1}{2} \log(\frac{1}{0.3} - 1) \approx 0.42 \Rightarrow H(\mathbf{x}) = \text{sign}\,(\alpha_1 h_1(\mathbf{x}))$$

Ronda 2



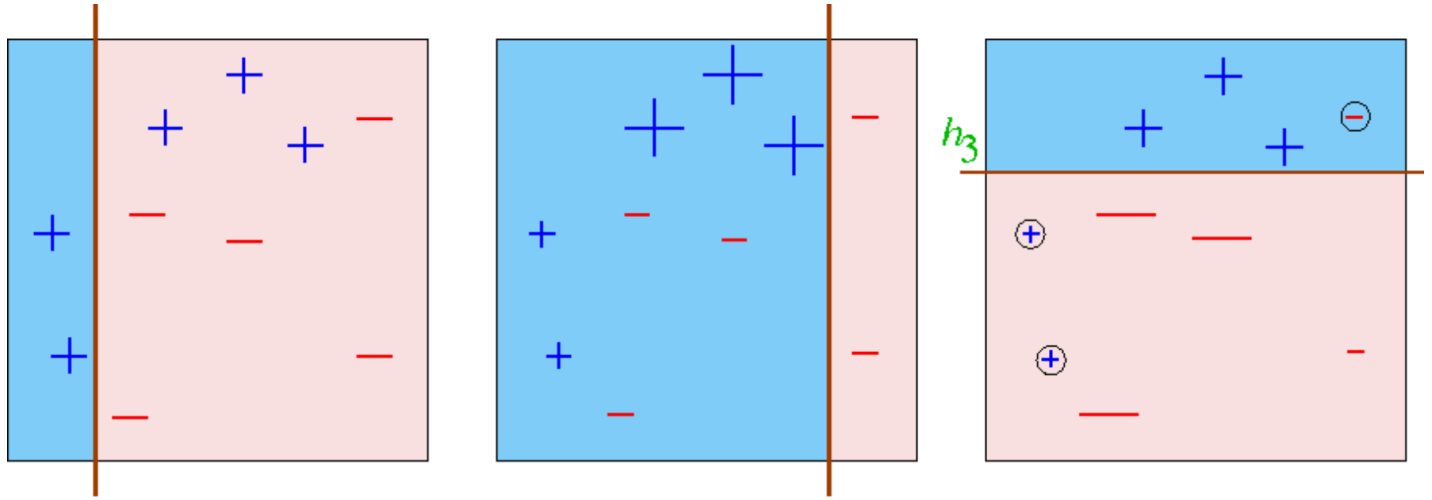
$\varepsilon_2 = 0.21$

$\alpha_2 = 0.65$

$$\mathbf{w} = \text{updated weights} \Rightarrow \text{Train a classifier (using } \mathbf{w}) \Rightarrow \text{err}_2 = \frac{\sum_{i=1}^{10} w_i \mathbb{I}\{h_2(\mathbf{x}^{(i)}) \neq t^{(i)}\}}{\sum_{i=1}^{N} w_i} = 0.21$$

$$\Rightarrow \alpha_2 = \frac{1}{2} \log \frac{1 - \text{err}_3}{\text{err}_3} = \frac{1}{2} \log(\frac{1}{0.21} - 1) \approx 0.66 \Rightarrow H(\mathbf{x}) = \text{sign}\,(\alpha_1 h_1(\mathbf{x}) + \alpha_2 h_2(\mathbf{x}))$$

Ronda 3



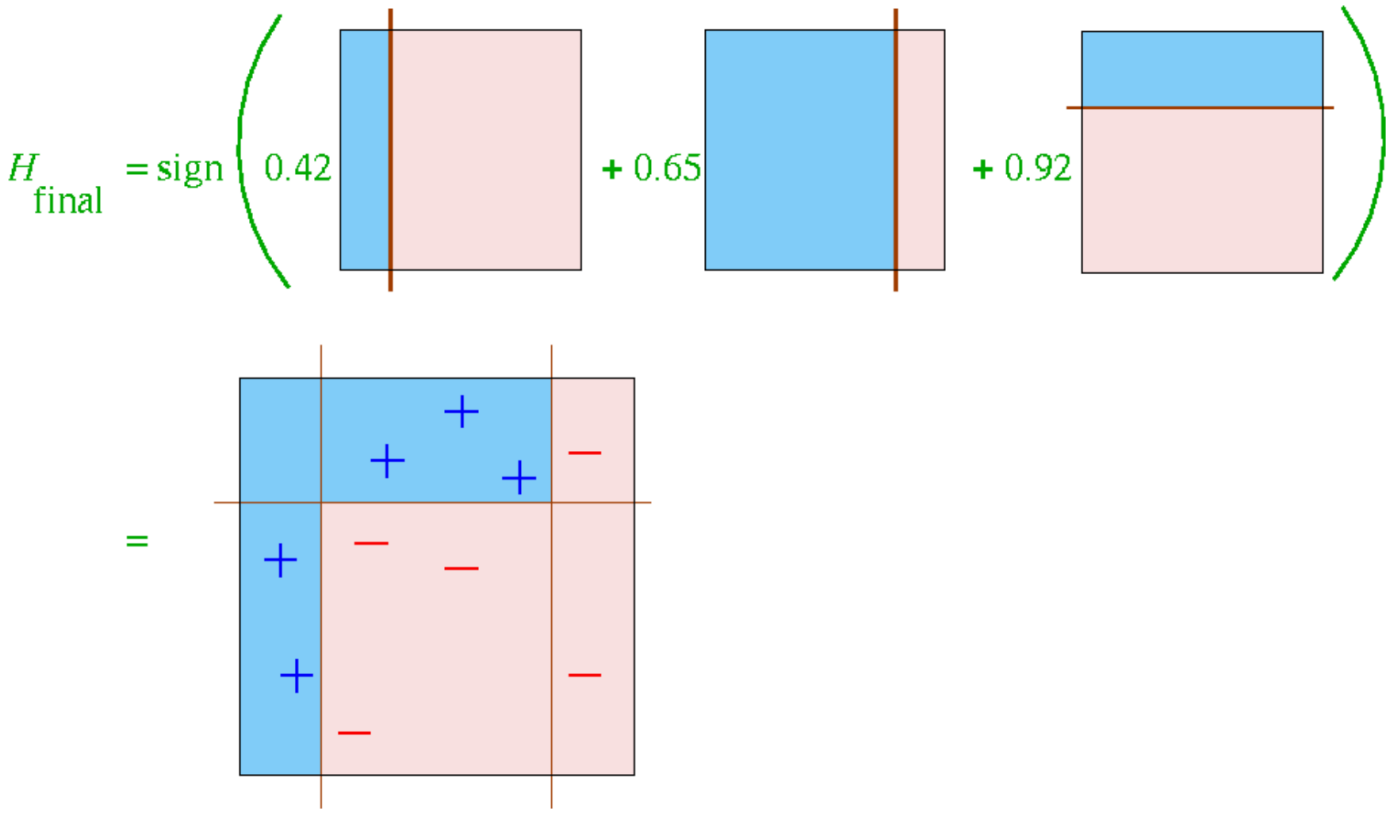
$\varepsilon_3 = 0.14$

$\alpha_3 = 0.92$

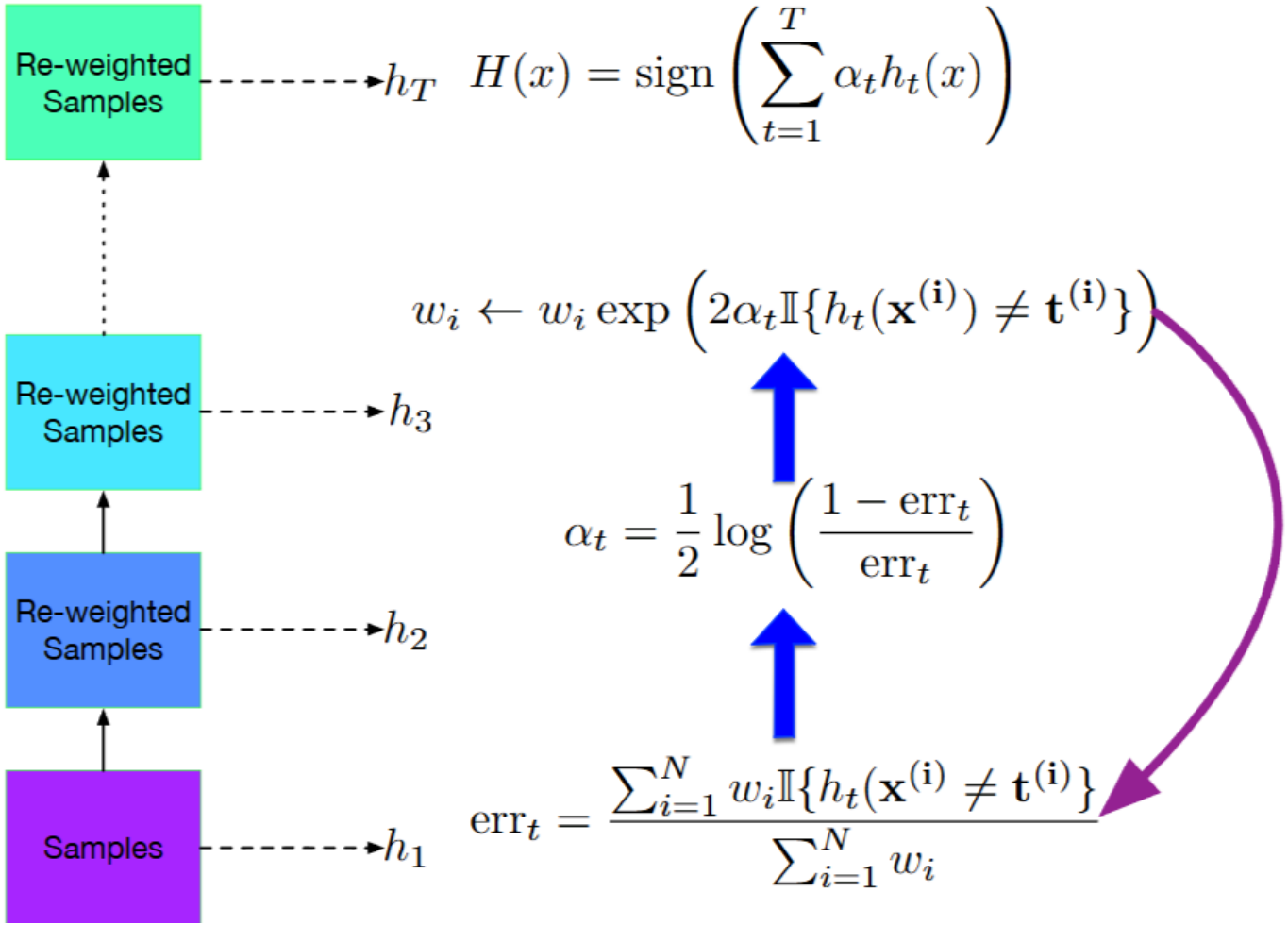
$$\mathbf{w} = \text{updated weights} \Rightarrow \text{Train a classifier (using } \mathbf{w}) \Rightarrow \text{err}_3 = \frac{\sum_{i=1}^{10} w_i \mathbb{I}\{h_3(\mathbf{x}^{(i)}) \neq t^{(i)}\}}{\sum_{i=1}^{N} w_i} = 0.14$$

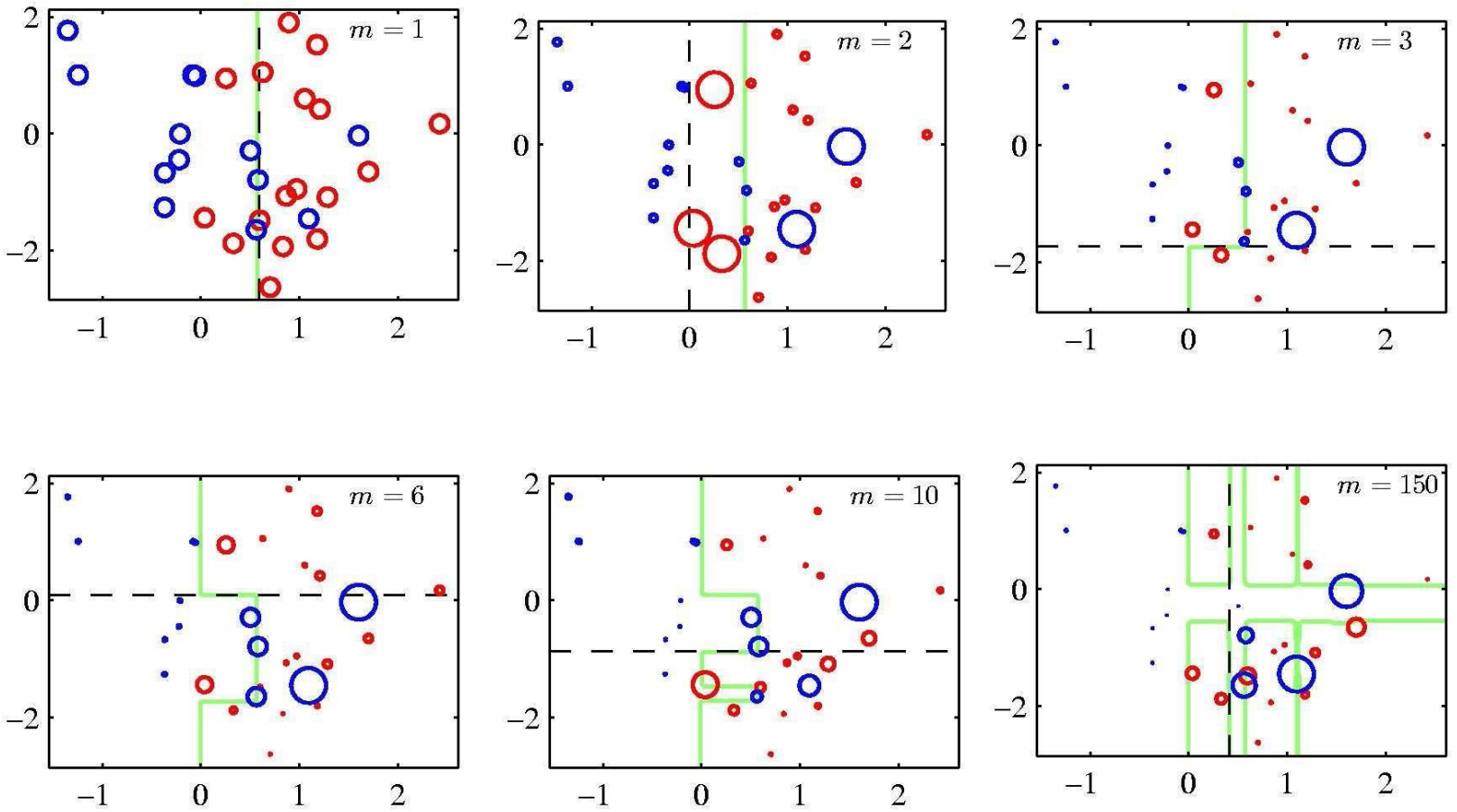
$$\Rightarrow \alpha_3 = \frac{1}{2} \log \frac{1 - \text{err}_3}{\text{err}_3} = \frac{1}{2} \log\left(\frac{1}{0.14} - 1\right) \approx 0.91 \Rightarrow H(\mathbf{x}) = \text{sign}\left(\alpha_1 h_1(\mathbf{x}) + \alpha_2 h_2(\mathbf{x}) + \alpha_3 h_3(\mathbf{x})\right)$$

Clasificador final



Algoritmo AdaBoost





Cada figura muestra el número m de modelos base entrenados hasta ese momento, la decisión del modelo más reciente (negro discontinuo) y el límite del conjunto (verde)

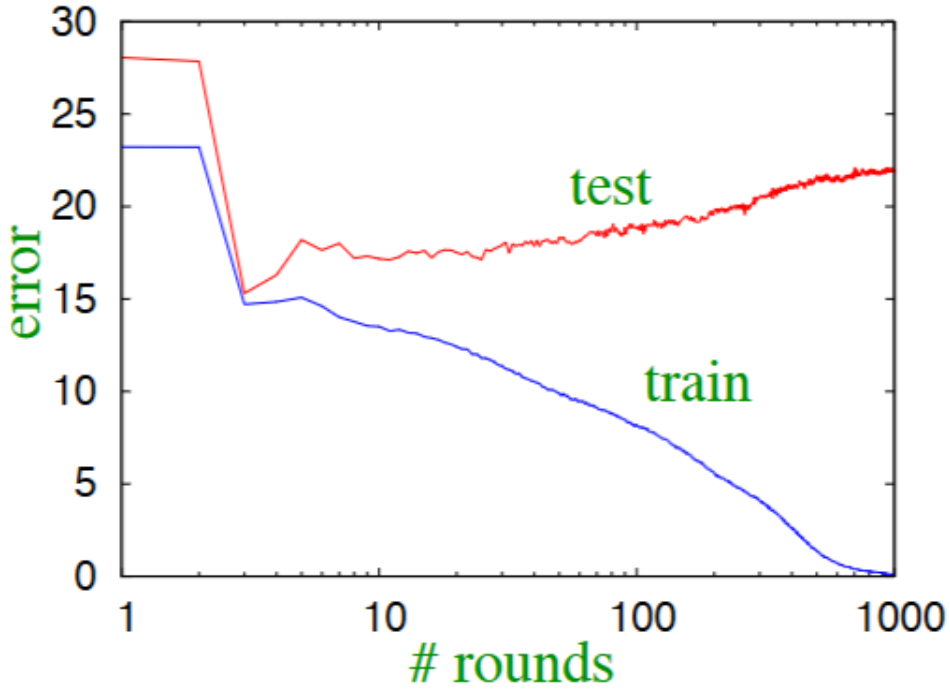
Error de generalización de AdaBoost

El error de entrenamiento (pérdida) de AdaBoost converge a cero. ¿Qué ocurre con el error de prueba de H?

A medida que añadimos más clasificadores débiles, el clasificador global H se vuelve más "complejo".

Esperamos que los clasificadores más complejos se ajusten en exceso.

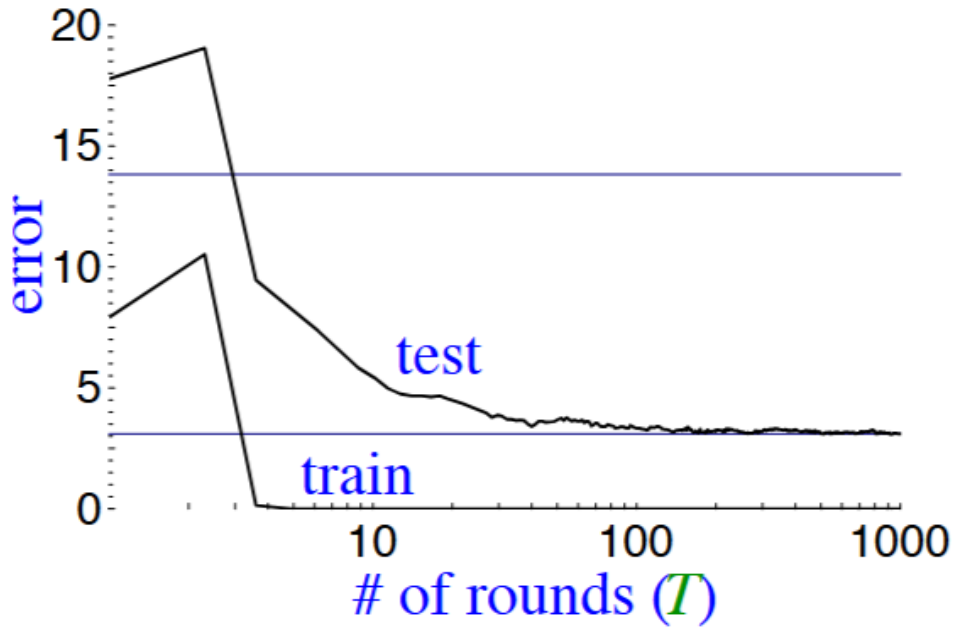
Si se ejecuta AdaBoost el tiempo suficiente, puede, de hecho, sobreajustarse.



Error de generalización de AdaBoost

Pero a menudo no es así.

A veces, ¡el error de prueba disminuye incluso después de que el error de entrenamiento sea cero!





Resumen del Boosting

El boosting reduce el sesgo generando un conjunto de clasificadores débiles.

Cada clasificador se entrena para reducir los errores del conjunto anterior.

Es bastante resistente al sobreajuste, aunque puede sobreajustarse.

El punto de vista de minimización de pérdidas de AdaBoost nos permite derivar otros algoritmos de boosting para regresión, clasificación, etc.



Resumen de Ensambles

Los ensambles combinan clasificadores para mejorar el rendimiento

Boosting

- Reduce el sesgo
- Aumenta la varianza (un conjunto grande puede provocar un sobreajuste)
- Secuencial

Bagging

- Reduce la varianza (un conjunto grande no puede causar sobreajuste)
- El sesgo no se modifica (mucho)
- En paralelo
- Desea minimizar la correlación entre los elementos del conjunto.

Problema: Clasificación con AdaBoost

Clasificadores débiles: Usaremos decisión stumps (tocones de decisión), que son árboles de profundidad 1 con la forma:

$$h(x) = \begin{cases} +1 & \text{si } x \leq \theta \\ -1 & \text{si } x > \theta \end{cases} \quad \text{ó} \quad h(x) = \begin{cases} -1 & \text{si } x \leq \theta \\ +1 & \text{si } x > \theta \end{cases}$$

donde θ es un umbral.

Se realizarán **3 iteraciones** de AdaBoost.

Se tiene el siguiente conjunto de datos con 10 puntos en $\mathbb{R}^1$ (una dimensión):

Punto i	x_i	**Clase** y_i
1	1	+1
2	2	+1
3	3	-1
4	4	-1
5	5	-1
6	6	+1
7	7	+1
8	8	+1
9	9	-1
10	10	-1

Parte A

1. Inicialice los pesos para cada punto. En AdaBoost, los pesos iniciales son uniformes.
2. ¿Cuál es la suma total de los pesos? ¿Por qué es importante que sumen 1?

Parte A (respuesta)

1. Pesos iniciales:

$$w_i^{(1)} = \frac{1}{N} = \frac{1}{10} = 0.1 \quad \text{para todo } i$$

i	1	2	3	4	5	6	7	8	9	10
$w_i^{(1)}$	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1

2. Suma total de pesos:

$$\sum_{i=1}^{10} w_i^{(1)} = 10 \times 0.1 = \boxed{1.0}$$

Parte B

Considere los siguientes decision stumps candidatos:

Stump	Regla	Descripción
h_1	$+1$ si $x \leq 2.5$, sino -1	Divide entre 2 y 3
h_2	$+1$ si $x \leq 5.5$, sino -1	Divide entre 5 y 6
h_3	-1 si $x \leq 5.5$, sino $+1$	Divide entre 5 y 6 (invertido)
h_4	$+1$ si $x \leq 8.5$, sino -1	Divide entre 8 y 9

1. Para cada stump, determine las predicciones para todos los puntos.
2. Calcule el error ponderado de cada stump
3. ¿Cuál stump tiene el menor error ponderado?

Parte B (solución)

Predicciones de cada stump:

Stump h_1: "+1 si $x \leq 2.5$, sino -1"

i	x_i	y_i	$h_1(x_i)$	¿Error?	$w_i^{(1)}$
1	1	+1	+1	No	0.1
2	2	+1	+1	No	0.1
3	3	-1	-1	No	0.1
4	4	-1	-1	No	0.1
5	5	-1	-1	No	0.1
6	6	+1	-1	**Sí**	0.1
7	7	+1	-1	**Sí**	0.1
8	8	+1	-1	**Sí**	0.1
9	9	-1	-1	No	0.1
10	10	-1	-1	No	0.1

$$\epsilon_{h_1} = 0.1 + 0.1 + 0.1 = \boxed{0.3}$$

Stump h_2: "+1 si $x \leq 5.5$, sino -1"

i	x_i	y_i	$h_2(x_i)$	¿Error?
1	1	+1	+1	No
2	2	+1	+1	No
3	3	-1	+1	**Sí**
4	4	-1	+1	**Sí**
5	5	-1	+1	**Sí**
6	6	+1	-1	**Sí**
7	7	+1	-1	**Sí**
8	8	+1	-1	**Sí**
9	9	-1	-1	No
10	10	-1	-1	No

$$\epsilon_{h_2} = 6 \times 0.1 = \boxed{0.6}$$

Parte B (solución)

Stump h_3: "-1 si $x \leq 5.5$, sino +1"

i	x_i	y_i	$h_3(x_i)$	¿Error?
1	1	+1	-1	**Sí**
2	2	+1	-1	**Sí**
3	3	-1	-1	No
4	4	-1	-1	No
5	5	-1	-1	No
6	6	+1	+1	No
7	7	+1	+1	No
8	8	+1	+1	No
9	9	-1	+1	**Sí**
10	10	-1	+1	**Sí**

$$\epsilon_{h_3} = 4 \times 0.1 = \boxed{0.4}$$

Stump h_4: "+1 si $x \leq 8.5$, sino -1"

i	x_i	y_i	$h_4(x_i)$	¿Error?
1	1	+1	+1	No
2	2	+1	+1	No
3	3	-1	+1	**Sí**
4	4	-1	+1	**Sí**
5	5	-1	+1	**Sí**
6	6	+1	+1	No
7	7	+1	+1	No
8	8	+1	+1	No
9	9	-1	-1	No
10	10	-1	-1	No

$$\epsilon_{h_4} = 3 \times 0.1 = \boxed{0.3}$$

Parte B (solución)

Resumen de errores:

Stump	Error ϵ
h_1	0.3
h_2	0.6
h_3	0.4
h_4	0.3

Mejor stump: h_1 o h_4 (empate con $\epsilon = 0.3$).

$$\boxed{\text{Elegimos } h_1 \text{ (o } h_4\text{): } \epsilon_1 = 0.3}$$

Parte C

Una vez seleccionado el mejor stump con error ϵ1

1. Calcule el peso del clasificador α1}
2. ¿Qué significa que α1>0? ¿Qué pasaría si ϵ1=0.5?

Actualice los pesos para la siguiente iteración:

$$w_i^{(2)} = \frac{w_i^{(1)} \cdot \exp(-\alpha_1 \cdot y_i \cdot h_1(x_i))}{Z_1}$$

donde Z_1 es el factor de normalización para que los pesos sumen 1.

Parte C (solución)

Cálculo del peso α_1:

$$\alpha_1 = \frac{1}{2}\ln\left(\frac{1-\epsilon_1}{\epsilon_1}\right) = \frac{1}{2}\ln\left(\frac{1-0.3}{0.3}\right) = \frac{1}{2}\ln\left(\frac{0.7}{0.3}\right)$$

$$\alpha_1 = \frac{1}{2}\ln(2.333) = \frac{1}{2}(0.847)$$

$$\boxed{\alpha_1 \approx 0.424}$$

Interpretación:

- $\alpha_1 > 0$ significa que el clasificador es mejor que el azar (error < 0.5)
- Si $\epsilon_1 = 0.5$, entonces $\alpha_1 = \frac{1}{2}\ln(1) = 0$, y el clasificador no contribuiría (es equivalente a adivinar al azar)

La fórmula es:

$$w_i^{(2)} = w_i^{(1)} \cdot \exp(-\alpha_1 \cdot y_i \cdot h_1(x_i))$$

Recordemos:

- Si clasificación **correcta:** $y_i \cdot h_1(x_i) = +1 \rightarrow$ multiplicamos por $e^{-\alpha_1} \approx e^{-0.424} \approx 0.654$
- Si clasificación **incorrecta:** $y_i \cdot h_1(x_i) = -1 \rightarrow$ multiplicamos por $e^{+\alpha_1} \approx e^{0.424} \approx 1.528$

Parte C (solución)

i	y_i	$h_1(x_i)$	$y_i \cdot h_1$	$w_i^{(1)}$	Factor	$\tilde{w}_i^{(2)}$ (sin normalizar)
1	+1	+1	+1 (correcto)	0.1	0.654	0.0654
2	+1	+1	+1 (correcto)	0.1	0.654	0.0654
3	-1	-1	+1 (correcto)	0.1	0.654	0.0654
4	-1	-1	+1 (correcto)	0.1	0.654	0.0654
5	-1	-1	+1 (correcto)	0.1	0.654	0.0654
6	+1	-1	-1 (incorrecto)	0.1	1.528	0.1528
7	+1	-1	-1 (incorrecto)	0.1	1.528	0.1528
8	+1	-1	-1 (incorrecto)	0.1	1.528	0.1528
9	-1	-1	+1 (correcto)	0.1	0.654	0.0654
10	-1	-1	+1 (correcto)	0.1	0.654	0.0654

Factor de normalización:

$$Z_1 = 7 \times 0.0654 + 3 \times 0.1528 = 0.4578 + 0.4584 = 0.9162$$

Pesos normalizados:

i	$\tilde{w}_i^{(2)}$	$w_i^{(2)} = \tilde{w}/Z_1$
1	0.0654	**0.0714**
2	0.0654	**0.0714**
3	0.0654	**0.0714**
4	0.0654	**0.0714**
5	0.0654	**0.0714**
6	0.1528	**0.1667**
7	0.1528	**0.1667**
8	0.1528	**0.1667**
9	0.0654	**0.0714**
10	0.0654	**0.0714**

Verificación: $7 \times 0.0714 + 3 \times 0.1667 = 0.4998 + 0.5001 \approx 1.0$ ✓

Observación: Los puntos mal clasificados (6, 7, 8) ahora tienen peso 0.1667 > 0.1, mientras que los correctos tienen peso 0.0714 < 0.1.

$$w_i^{(2)} = \begin{cases} 0.0714 & \text{para } i \in \{1,2,3,4,5,9,10\} \text{ (correctos)} \\ 0.1667 & \text{para } i \in \{6,7,8\} \text{ (incorrectos)} \end{cases}$$

Parte D

Considerar los 3 clasificadores y usar el clasificador final de AdaBoost :

$$H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t \cdot h_t(x)\right)$$

- h_1: "+1 si $x \leq 2.5$, sino -1" con $\alpha_1 = 0.424$
- h_2: "+1 si $x \leq 8.5$, sino -1" con $\alpha_2 = 0.650$
- h_3: "-1 si $x \leq 5.5$, sino +1" con $\alpha_3 = 0.751$

Se pide:

1. Escriba la expresión explícita de $H(x)$ con los tres clasificadores y sus pesos $\alpha_1, \alpha_2, \alpha_3$.
2. Calcule $H(x_i)$ para cada punto de entrenamiento y determine el **error de entrenamiento** del clasificador final. Complete la tabla:

i	x_i	y_i	$\alpha_1 h_1$	$\alpha_2 h_2$	$\alpha_3 h_3$	**Suma**	$H(x_i)$	**¿Correcto?**
1	1	+1						
2	2	+1						
...	...	...						

3. Compare el error de entrenamiento del clasificador final con los errores individuales de cada stump. ¿Qué observa?

Parte D (solución)

1. Expresión del clasificador final:

Con los tres clasificadores:

- h_1: "+1 si $x \leq 2.5$, sino -1" con $\alpha_1 = 0.424$
- h_2: "+1 si $x \leq 8.5$, sino -1" con $\alpha_2 = 0.650$
- h_3: "-1 si $x \leq 5.5$, sino +1" con $\alpha_3 = 0.751$

$$\boxed{H(x) = \text{sign}(0.424 \cdot h_1(x) + 0.650 \cdot h_2(x) + 0.751 \cdot h_3(x))}$$

Parte D (solución)

2. Evaluación para cada punto:

i	x_i	y_i	h_1	h_2	h_3	$0.424h_1$	$0.650h_2$	$0.751h_3$	**Suma**	$H(x_i)$	**¿Correcto?**
1	1	+1	+1	+1	-1	+0.424	+0.650	-0.751	+0.323	+1	✓
2	2	+1	+1	+1	-1	+0.424	+0.650	-0.751	+0.323	+1	✓
3	3	-1	-1	+1	-1	-0.424	+0.650	-0.751	-0.525	-1	✓
4	4	-1	-1	+1	-1	-0.424	+0.650	-0.751	-0.525	-1	✓
5	5	-1	-1	+1	-1	-0.424	+0.650	-0.751	-0.525	-1	✓
6	6	+1	-1	+1	+1	-0.424	+0.650	+0.751	+0.977	+1	✓
7	7	+1	-1	+1	+1	-0.424	+0.650	+0.751	+0.977	+1	✓
8	8	+1	-1	+1	+1	-0.424	+0.650	+0.751	+0.977	+1	✓
9	9	-1	-1	-1	+1	-0.424	-0.650	+0.751	-0.323	-1	✓
10	10	-1	-1	-1	+1	-0.424	-0.650	+0.751	-0.323	-1	✓

Error de entrenamiento:

$$\text{Error de entrenamiento} = \frac{0}{10} = 0\%$$

Parte D (solución)

3. Comparación de errores:

Clasificador	Error individual
h_1 solo	30% (3 errores)
h_2 solo	30% (3 errores)
h_3 solo	40% (4 errores)
$H(x)$ **combinado**	**0%**

Ningún clasificador individual puede separar perfectamente los datos, pero la combinación ponderada sí puede. Esta es la esencia del boosting.

Parte E

Se pide responder:

1. ¿Por qué AdaBoost se llama "adaptativo"? (1 punto)
2. Si un clasificador débil tiene error $\epsilon_t > 0.5$, ¿qué sucede con α_t? ¿Tiene sentido incluir este clasificador? (2 puntos)
3. ¿Cuál es la principal diferencia entre **Bagging** (usado en Random Forest) y **Boosting** (usado en AdaBoost)? (2 puntos)

Parte E (solución)

2. Si $\epsilon_t > 0.5$, ¿qué pasa con α_t?

$$\alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$$

Si $\epsilon_t > 0.5$, entonces $\frac{1-\epsilon_t}{\epsilon_t} < 1$, y:

$$\ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right) < 0 \quad \Rightarrow \quad \alpha_t < 0$$

$\alpha_t < 0$ significa que el clasificador se incluye con signo invertido.

Esto tiene sentido: si un clasificador tiene más del 50% de error, es peor que adivinar al azar, así que AdaBoost invierte sus predicciones (lo cual lo convierte en un clasificador con error < 50%).

En la práctica, la mayoría de implementaciones detienen el algoritmo si $\epsilon_t \geq 0.5$.



Preguntas Vibe Quest

- Explora escenarios donde los métodos de ensamble NO son la mejor opción. ¿Existen tipos de datos, problemas o contextos donde un modelo simple supere consistentemente a los ensambles? Diseña experimentos para identificar estos "puntos de quiebre" y propón explicaciones teóricas.
- ¿Cuántos modelos base necesita realmente un ensamble? Investiga cómo determinar el número óptimo de estimadores considerando múltiples factores: rendimiento, tiempo de cómputo, memoria, y estabilidad. ¿Existe una fórmula universal o depende completamente del contexto?
- En los ensambles existe una tensión entre tener modelos diversos (que cometan errores diferentes) versus modelos precisos individualmente. ¿Cómo medirías y optimizarías esta tensión? ¿Qué estrategias creativas existen para maximizar ambas características?
- Los ensambles clásicos asumen datos estáticos, pero el mundo real es dinámico. ¿Cómo adaptarías Random Forest o AdaBoost para problemas donde los datos cambian constantemente? Propón y evalúa estrategias para "ensambles adaptativos" que evolucionen con el tiempo.